Submission Template for

INFS2200/INFS7903 Assignment 2

Semester 2, 2026

Submission of Screenshot for Assignment 2

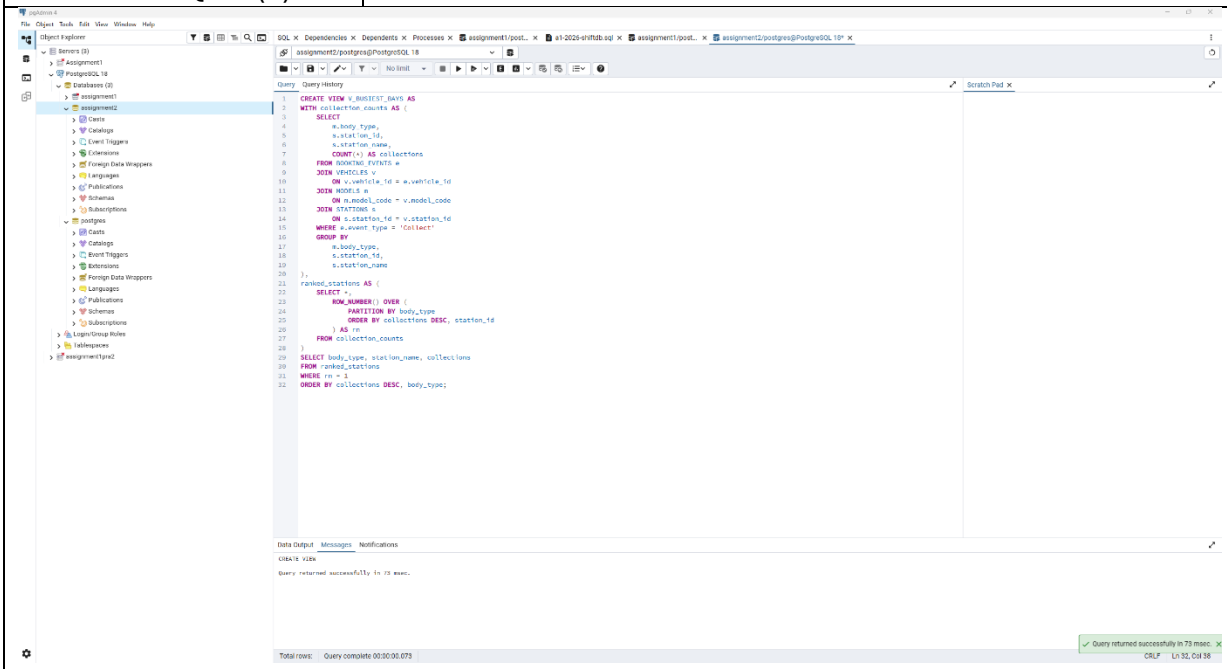
Submitted by:

Student Name	Student ID	Practical Session	Tutor Name
Yuyang Gong	49079984	Friday 9:00–10:00	Mr. Li, Jiayun

About AI Usage:

Did you use AI in this assignment?	Yes
The purpose of AI used in this assignment?	To help draft and debug SQL, interpret PostgreSQL output, and complete and format the report.
To what extend that AI is used in this submitted work?	AI assisted with substantial SQL and explanatory text, checked outputs using an isolated local replay, and supplemented incomplete captures. The original interactive screenshots and hand-drawn graph were supplied by the student.
Did you discuss your AI usage with the tutor during the "*Show to Tutor*" Lab sessions?	No. A2 has not yet been shown to the tutor, and this AI use has not yet been discussed.

Screenshot Q1.1 (1)	Marks: /1



Explanation

The first CTE counts Collect events for each body type and station. ROW_NUMBER partitions those counts by body type and picks the largest count; station_id resolves ties deterministically so exactly one station is returned per represented body type. The final ORDER BY puts the largest winner first.

The schema records a station on VEHICLES, rather than on BOOKING_EVENTS. The station reported is therefore the vehicle's recorded station, which is the location available in the supplied schema.

Comments:

Screenshot Q1.1(2)	Marks: /0.5

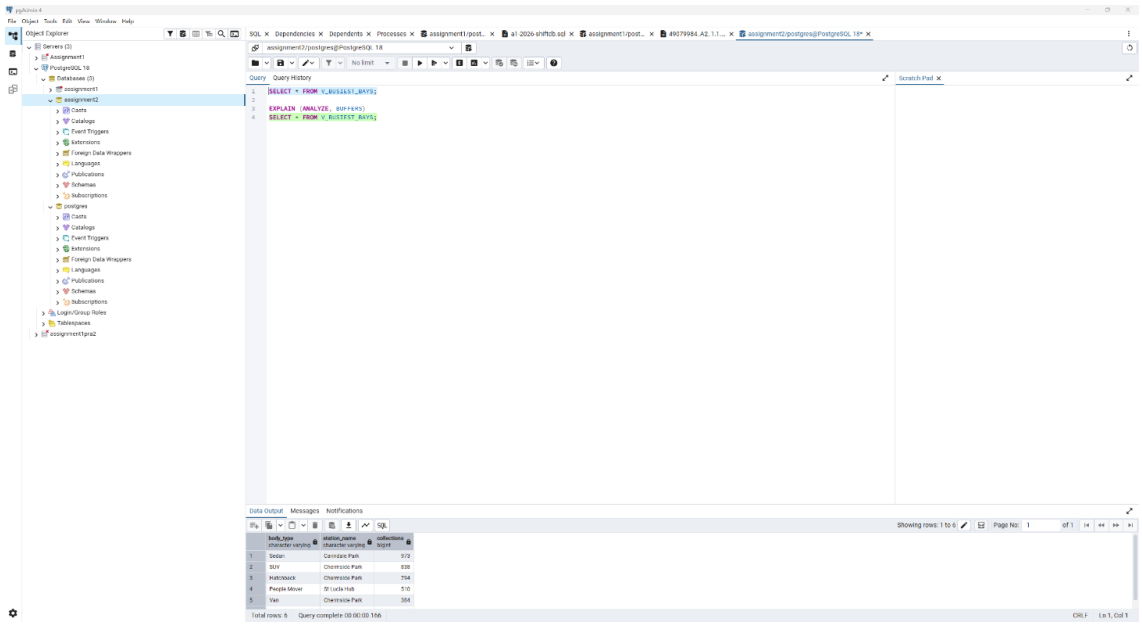
Evidence note: the newly supplied native pgAdmin captures are used in Q1.1.2, Q1.3.2, Q2.1.2 and Q3.1.1. Other plans labelled as replay output come from the previously validated clean PostgreSQL 18.6 replay. A separately labelled complete replay plan supplements Query A in Q1.3.2 because its native capture clips two long conditions. Measurements belong to their labelled runs; captures from different stages are not a controlled timing experiment. Original Q3.2 and Q4 interactive evidence is retained.

Screenshot Q1.1(2)	Marks: /0.5

SQL

```
SELECT * FROM V_BUSIEST_BAYS;

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM V_BUSIEST_BAYS;
```



Screenshot Q1.1(2)	Marks: /0.5

Execution plan without IDX_EVENT_TYPE_TIME, native pgAdmin, lines 1 to 34

	QUERY PLAN text
1	Sort (cost=4786.08..4786.08 rows=2 width=32) (actual time=47.879..52.128 rows=6.00 loops=1)
2	Sort Key: ranked_stations.collections DESC, ranked_stations.body_type
3	Sort Method: quicksort Memory: 25kB
4	Buffers: shared hit=1667
5	-> Subquery Scan on ranked_stations (cost=4686.49..4786.07 rows=2 width=32) (actual time=47.652..52.118 rows=6.00 loops=1)
6	Filter: (ranked_stations.rn = 1)
7	Buffers: shared hit=1664
8	-> WindowAgg (cost=4686.49..4781.57 rows=360 width=47) (actual time=47.651..52.116 rows=6.00 loops=1)
9	Window: w1 AS (PARTITION BY collection_counts.body_type ORDER BY collection_counts.collections, collection_counts.station_id ROWS UNBOUNDED PRECEDING)
10	Run Condition: (row_number() OVER w1 <= 1)
11	Storage: Memory Maximum Storage: 17kB
12	Buffers: shared hit=1664
13	-> Incremental Sort (cost=4686.23..4774.37 rows=360 width=39) (actual time=47.640..52.093 rows=279.00 loops=1)
14	Sort Key: collection_counts.body_type, collection_counts.collections DESC, collection_counts.station_id
15	Presorted Key: collection_counts.body_type
16	Full-sort Groups: 6 Sort Method: quicksort Average Memory: 28kB Peak Memory: 28kB
17	Buffers: shared hit=1664
18	-> Subquery Scan on collection_counts (cost=4685.83..4763.77 rows=360 width=39) (actual time=47.589..52.009 rows=279.00 loops=1)
19	Buffers: shared hit=1664
20	-> Finalize GroupAggregate (cost=4685.83..4763.77 rows=360 width=39) (actual time=47.588..51.992 rows=279.00 loops=1)
21	Group Key: m.body_type, s.station_id
22	Buffers: shared hit=1664
23	-> Gather Merge (cost=4685.83..4755.58 rows=612 width=39) (actual time=47.582..51.930 rows=558.00 loops=1)
24	Workers Planned: 1
25	Workers Launched: 1
26	Buffers: shared hit=1664
27	-> Sort (cost=3685.82..3686.72 rows=360 width=39) (actual time=27.239..27.248 rows=279.00 loops=2)
28	Sort Key: m.body_type, s.station_id
29	Sort Method: quicksort Memory: 41kB
30	Buffers: shared hit=1664
31	Worker 0: Sort Method: quicksort Memory: 41kB
32	-> Partial HashAggregate (cost=3666.93..3670.53 rows=360 width=39) (actual time=26.755..26.773 rows=279.00 loops=2)
33	Group Key: m.body_type, s.station_id
34	Batches: 1 Memory Usage: 72kB

Execution plan without IDX_EVENT_TYPE_TIME, native pgAdmin, lines 35 to 68

	QUERY PLAN text
35	Buffers: shared hit=1656
36	Worker 0: Batches: 1 Memory Usage: 72kB
37	-> Hash Join (cost=32.62..3404.07 rows=35048 width=31) (actual time=0.838..23.307 rows=29769.50 loops=2)
38	Hash Cond: ((v.station_id)::text = (s.station_id)::text)
39	Buffers: shared hit=1656
40	-> Hash Join (cost=30.27..3303.22 rows=35048 width=14) (actual time=0.612..20.362 rows=29769.50 loops=2)
41	Hash Cond: ((v.model_code)::text = (m.model_code)::text)
42	Buffers: shared hit=1654
43	-> Hash Join (cost=27.25..3203.87 rows=35048 width=15) (actual time=0.435..17.595 rows=29769.50 loops=2)
44	Hash Cond: ((e.vehicle_id)::text = (v.vehicle_id)::text)
45	Buffers: shared hit=1652
46	-> Parallel Seq Scan on booking_events e (cost=0.00..3084.18 rows=35048 width=9) (actual time=0.143..14.286 rows=29769.50 loops=2)
47	Filter: ((event_type)::text = 'Collect'::text)
48	Rows Removed by Filter: 68453
49	Buffers: shared hit=1638
50	-> Hash (cost=16.00..16.00 rows=900 width=24) (actual time=0.266..0.266 rows=900.00 loops=2)
51	Buckets: 1024 Batches: 1 Memory Usage: 58kB
52	Buffers: shared hit=14
53	-> Seq Scan on vehicles v (cost=0.00..16.00 rows=900 width=24) (actual time=0.125..0.194 rows=900.00 loops=2)
54	Buffers: shared hit=14
55	-> Hash (cost=1.90..1.90 rows=90 width=15) (actual time=0.171..0.171 rows=90.00 loops=2)
56	Buckets: 1024 Batches: 1 Memory Usage: 13kB
57	Buffers: shared hit=2
58	-> Seq Scan on models m (cost=0.00..1.90 rows=90 width=15) (actual time=0.158..0.162 rows=90.00 loops=2)
59	Buffers: shared hit=2
60	-> Hash (cost=1.60..1.60 rows=60 width=24) (actual time=0.219..0.220 rows=60.00 loops=2)
61	Buckets: 1024 Batches: 1 Memory Usage: 12kB
62	Buffers: shared hit=2
63	-> Seq Scan on stations s (cost=0.00..1.60 rows=60 width=24) (actual time=0.205..0.207 rows=60.00 loops=2)
64	Buffers: shared hit=2
65	Planning:
66	Buffers: shared hit=203 dirtied=1
67	Planning Time: 7.797 ms
68	Execution Time: 52.686 ms

Screenshot Q1.1(2)	Marks: /0.5

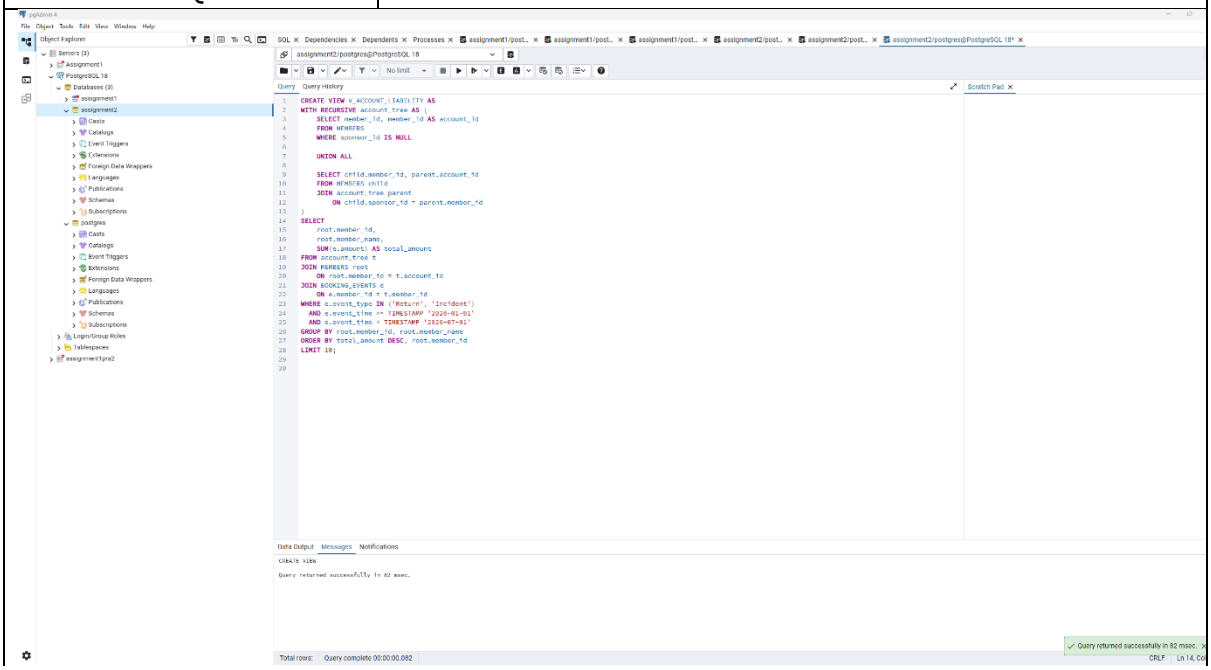
Explanation

The view returns six rows. Sedan at Carindale Park leads with 973 collections, followed by SUV 838, Hatchback 794, People Mover 510, Van 384 and Ute 333. In the native repeat measurement without IDX_EVENT_TYPE_TIME, the top node estimates 2 rows and returns 6, with cost 4786.08..4786.08. Planning takes 7.797 ms and execution takes 52.686 ms.

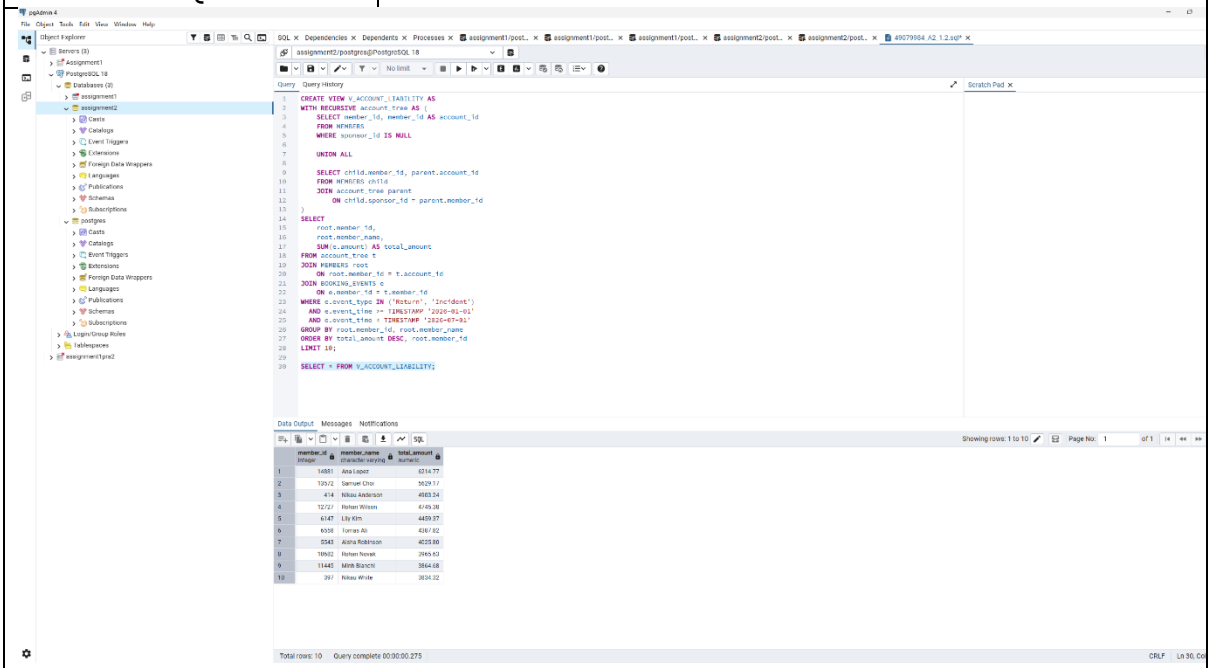
A Parallel Seq Scan filters BOOKING_EVENTS to Collect events. Its 29,769.5 actual rows per loop across two loops represent 59,539 collections. Hash joins attach vehicle, model and station attributes. Partial aggregation and Finalize GroupAggregate produce 279 body-type/station groups in this captured run; incremental sorting and WindowAgg select one winner per body type, and the final sort orders the six winners. The small final result does not avoid processing the underlying event log.

Comments:

Screenshot Q1.2	Marks: /1.5



Screenshot Q1.2	Marks: /1.5



```
CREATE VIEW V_ACCOUNT_LIABILITY AS
WITH RECURSIVE account_tree AS (
    SELECT member_id, member_id AS account_id
    FROM MEMBERS
    WHERE sponsor_id IS NULL

    UNION ALL

    SELECT child.member_id, parent.account_id
    FROM MEMBERS child
    JOIN account_tree parent
      ON child.sponsor_id = parent.member_id
)
SELECT
    root.member_id,
    root.member_name,
    SUM(e.amount) AS total_amount
FROM account_tree t
JOIN MEMBERS root
  ON root.member_id = t.account_id
JOIN BOOKING_EVENTS e
  ON e.member_id = t.member_id
WHERE e.event_type IN ('Return', 'Incident')
  AND e.event_time >= TIMESTAMP '2026-01-01'
  AND e.event_time < TIMESTAMP '2026-07-01'
GROUP BY root.member_id, root.member_name
ORDER BY total_amount DESC, root.member_id
LIMIT 10;

SELECT * FROM V_ACCOUNT_LIABILITY;
```

member_id integer	member_name character varying	total_amount numeric
1	14881	Ana Lopez	6314.77
2	10972	Samuel Choi	5629.17
3	414	Nikau Anderson	4983.24
4	12727	Rehan Wilson	4749.38
5	6147	Lily Kim	4459.37
6	6558	Tomas Ali	4367.92
7	5543	Aisha Robinson	4225.80
8	16802	Rehan Novak	3965.63
9	11445	Minh Bianchi	3864.68
10	397	Nikau White	3834.32

Explanation

The recursive seed maps each unsponsored account to itself. Each recursive step maps a sponsored child to the same root account as its parent, so every member in the supplied sponsorship forest maps to exactly one ultimately liable account. Joining charges to this mapping bills personal charges and every level of sponsored charges to the root, without counting an event at several intermediate accounts.

Only Return fares and Incident damage charges are included. The half-open timestamp range from 2026-01-01 up to, but excluding, 2026-07-01 includes all of 30 June. Totals are sorted descending, with member_id as a deterministic tie-breaker, and limited to ten accounts. The result is shown in Q1.3 Query A. An independent upward traversal produced the same ten rows.

Comments:

Screenshot Q1.3 (2)	Marks: /0.5

Q1.3 part 1 materialized view definition

```
CREATE MATERIALIZED VIEW MV_ACCOUNT_LIABILITY AS
WITH RECURSIVE account_tree AS (
    SELECT member_id, member_id AS account_id
    FROM MEMBERS
    WHERE sponsor_id IS NULL

    UNION ALL

    SELECT child.member_id, parent.account_id
    FROM MEMBERS child
    JOIN account_tree parent
        ON child.sponsor_id = parent.member_id
)
SELECT
    root.member_id,
    root.member_name,
    SUM(e.amount) AS total_amount
FROM account_tree t
JOIN MEMBERS root
    ON root.member_id = t.account_id
JOIN BOOKING_EVENTS e
    ON e.member_id = t.member_id
WHERE e.event_type IN ('Return', 'Incident')
  AND e.event_time >= TIMESTAMP '2026-01-01'
  AND e.event_time < TIMESTAMP '2026-07-01'
GROUP BY root.member_id, root.member_name
ORDER BY total_amount DESC, root.member_id
LIMIT 10;
```

Q1.3 part 2 SQL

```
-- Query A: retrieve the ordinary view results.
SELECT * FROM V_ACCOUNT_LIABILITY;

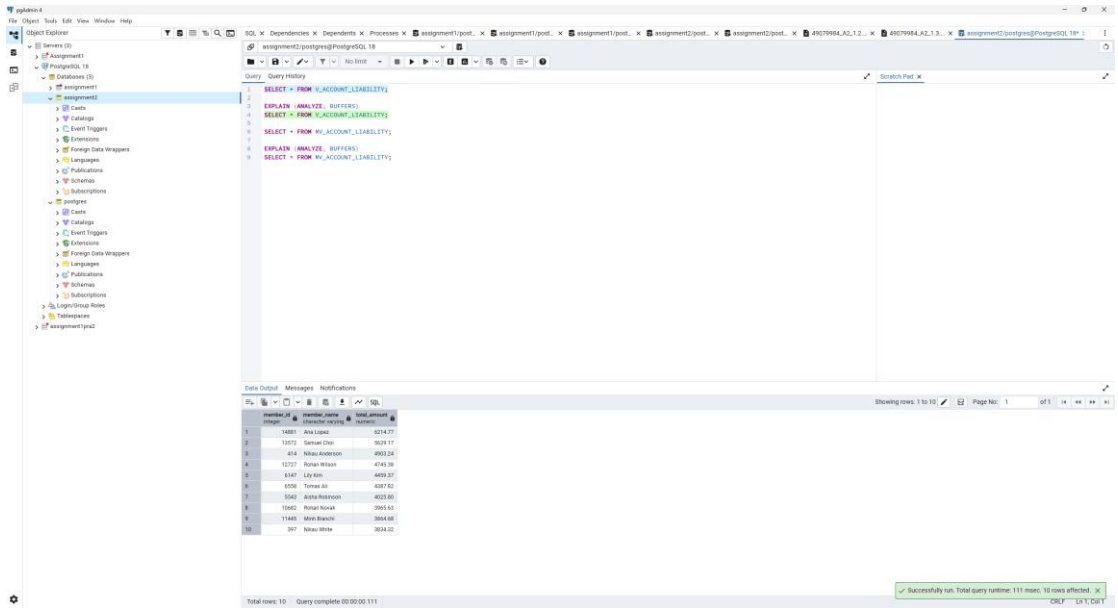
-- Query A: report the ordinary view execution plan.
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM V_ACCOUNT_LIABILITY;

-- Query B: retrieve the materialized view results.
SELECT * FROM MV_ACCOUNT_LIABILITY;

-- Query B: report the materialized view execution plan.
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM MV_ACCOUNT_LIABILITY;
```

Screenshot Q1.3 (2)	Marks: /0.5

Query A original result



Query A ordinary view, native pgAdmin, lines 1 to 34

	QUERY PLAN text
1	Limit (cost=370929.67..370929.69 rows=10 width=48) (actual time=44.446..44.453 rows=10.00 loops=1)
2	Buffers: shared hit=1769
3	CTE account_tree
4	-> Recursive Union (cost=0.00..102462.80 rows=1868540 width=8) (actual time=0.024..9.410 rows=24000.00 loops=1)
5	Storage: Memory Maximum Storage: 860kB
6	Buffers: shared hit=556
7	-> Seq Scan on members (cost=0.00..518.00 rows=15600 width=8) (actual time=0.021..1.411 rows=15600.00 loops=1)
8	Filter: (sponsor_id IS NULL)
9	Rows Removed by Filter: 8400
10	Buffers: shared hit=278
11	-> Hash Join (cost=818.00..8325.94 rows=185294 width=8) (actual time=0.645..1.589 rows=2100.00 loops=4)
12	Hash Cond: (parent.member_id = child.sponsor_id)
13	Buffers: shared hit=278
14	-> WorkTable Scan on account_tree parent (cost=0.00..3120.00 rows=156000 width=8) (actual time=0.001..0.160 rows=6000.00 loops=4)
15	-> Hash (cost=518.00..518.00 rows=24000 width=8) (actual time=2.469..2.470 rows=8400.00 loops=1)
16	Buckets: 32768 Batches: 1 Memory Usage: 585kB
17	Buffers: shared hit=278
18	-> Seq Scan on members child (cost=0.00..518.00 rows=24000 width=8) (actual time=0.027..1.397 rows=24000.00 loops=1)
19	Buffers: shared hit=278
20	-> Sort (cost=268466.87..268526.87 rows=24000 width=48) (actual time=44.445..44.449 rows=10.00 loops=1)
21	Sort Key: (sum(e.amount)) DESC, root.member_id
22	Sort Method: top-N heapsort Memory: 26kB
23	Buffers: shared hit=1769
24	-> HashAggregate (cost=267648.24..267948.24 rows=24000 width=48) (actual time=40.579..43.523 rows=10175.00 loops=1)
25	Group Key: root.member_id
26	Batches: 1 Memory Usage: 4625kB
27	Buffers: shared hit=1769
28	-> Hash Join (cost=4198.46..251537.16 rows=3222216 width=22) (actual time=8.591..32.947 rows=30276.00 loops=1)
29	Hash Cond: (t.member_id = e.member_id)
30	Buffers: shared hit=1769
31	-> Hash Join (cost=818.00..43094.59 rows=1868540 width=20) (actual time=2.874..19.526 rows=24000.00 loops=1)
32	Hash Cond: (t.account_id = root.member_id)
33	Buffers: shared hit=834
34	-> CTE Scan on account_tree t (cost=0.00..37370.80 rows=1868540 width=8) (actual time=0.026..12.015 rows=24000.00 loops=1)

Screenshot Q1.3 (2)	Marks: /0.5

Query A ordinary view, native pgAdmin, lines 21 to 54; overlap retained

	QUERY PLAN text
21	Sort Key: (sum(e.amount)) DESC, root.member_id
22	Sort Method: top-N heapsort Memory: 26kB
23	Buffers: shared hit=1769
24	-> HashAggregate (cost=267648.24..267948.24 rows=24000 width=48) (actual time=40.579..43.523 rows=10175.00 loops=1)
25	Group Key: root.member_id
26	Batches: 1 Memory Usage: 4625kB
27	Buffers: shared hit=1769
28	-> Hash Join (cost=4198.46..251537.16 rows=3222216 width=22) (actual time=8.591..32.947 rows=30276.00 loops=1)
29	Hash Cond: (t.member_id = e.member_id)
30	Buffers: shared hit=1769
31	-> Hash Join (cost=818.00..43094.59 rows=1868540 width=20) (actual time=2.874..19.526 rows=24000.00 loops=1)
32	Hash Cond: (t.account_id = root.member_id)
33	Buffers: shared hit=834
34	-> CTE Scan on account_tree t (cost=0.00..37370.80 rows=1868540 width=8) (actual time=0.026..12.015 rows=24000.00 loops=1)
35	Storage: Memory Maximum Storage: 1007kB
36	Buffers: shared hit=556
37	-> Hash (cost=518.00..518.00 rows=24000 width=16) (actual time=2.790..2.791 rows=24000.00 loops=1)
38	Buckets: 32768 Batches: 1 Memory Usage: 1400kB
39	Buffers: shared hit=278
40	-> Seq Scan on members root (cost=0.00..518.00 rows=24000 width=16) (actual time=0.004..1.138 rows=24000.00 loops=1)
41	Buffers: shared hit=278
42	-> Hash (cost=2999.87..2999.87 rows=30447 width=10) (actual time=5.666..5.667 rows=30276.00 loops=1)
43	Buckets: 32768 Batches: 1 Memory Usage: 1557kB
44	Buffers: shared hit=935
45	-> Bitmap Heap Scan on booking_events e (cost=829.05..2999.87 rows=30447 width=10) (actual time=0.616..3.690 rows=30276.00 loops=1)
46	Recheck Cond: (((event_type)::text = ANY ('{Return,Incident}'::text[])) AND (event_time >= '2026-01-01 00:00:00'::timestamp without time zone) AND (event_time < '2026-0...
47	Heap Blocks: exact=811
48	Buffers: shared hit=935
49	-> Bitmap Index Scan on idx_event_type_time (cost=0.00..821.44 rows=30447 width=0) (actual time=0.555..0.556 rows=30276.00 loops=1)
50	Index Cond: (((event_type)::text = ANY ('{Return,Incident}'::text[])) AND (event_time >= '2026-01-01 00:00:00'::timestamp without time zone) AND (event_time < '20...
51	Index Searches: 2
52	Buffers: shared hit=124
53	Planning Time: 0.384 ms
54	Execution Time: 45.543 ms

The native screenshots above clip the ends of two long filter conditions. The following complete plan is retained as supplementary evidence from the earlier replay. It is a separate run with its own estimates, access path and timing; it does not fill in unseen native output. Its execution time is 61.402 ms.

Screenshot Q1.3 (2)	Marks: /0.5

Supplementary complete ordinary-view plan from the earlier replay, part 1 of 2

Q1.3.2 statement 2 | 1 of 2

```
QUERY PLAN
------------------------------------------------------------------------
Limit  (cost=371952.28..371952.30 rows=10 width=48) (actual
  time=60.243..60.254 rows=10.00 loops=1)
 Buffers: shared hit=2476
 CTE account_tree
 ->  Recursive Union  (cost=0.00..102462.80 rows=1868540 width=8) (actual
   time=0.022..9.720 rows=24000.00 loops=1)
   Storage: Memory  Maximum Storage: 860kB
   Buffers: shared hit=556
   ->  Seq Scan on members  (cost=0.00..518.00 rows=15600 width=8) (actual
     time=0.019..1.625 rows=15600.00 loops=1)
     Filter: (sponsor_id IS NULL)
     Rows Removed by Filter: 8400
     Buffers: shared hit=278
   ->  Hash Join  (cost=818.00..8325.94 rows=185294 width=8) (actual
     time=0.757..1.618 rows=2100.00 loops=4)
     Hash Cond: (parent.member_id = child.sponsor_id)
     Buffers: shared hit=278
     ->  WorkTable Scan on account_tree parent  (cost=0.00..3120.00
       rows=156000 width=8) (actual time=0.001..0.170 rows=6000.00 loops=4)
     ->  Hash  (cost=518.00..518.00 rows=24000 width=8) (actual
       time=2.855..2.856 rows=8400.00 loops=1)
       Buckets: 32768  Batches: 1  Memory Usage: 585kB
       Buffers: shared hit=278
       ->  Seq Scan on members child  (cost=0.00..518.00 rows=24000 width=8)
         (actual time=0.034..1.640 rows=24000.00 loops=1)
         Buffers: shared hit=278
 ->  Sort  (cost=269489.48..269549.48 rows=24000 width=48) (actual
   time=60.241..60.248 rows=10.00 loops=1)
   Sort Key: (sum(e.amount)) DESC, root.member_id
   Sort Method: top-N heapsort  Memory: 26kB
   Buffers: shared hit=2476
   ->  HashAggregate  (cost=268670.85..268970.85 rows=24000 width=48) (actual
     time=55.844..59.227 rows=10175.00 loops=1)
     Group Key: root.member_id
```

Clean local replay | Actual psql output in a transcript viewer
Only whitespace and permitted long-result truncation changed

Screenshot Q1.3 (2)	Marks: /0.5

Supplementary complete ordinary-view replay plan, part 2 of 2

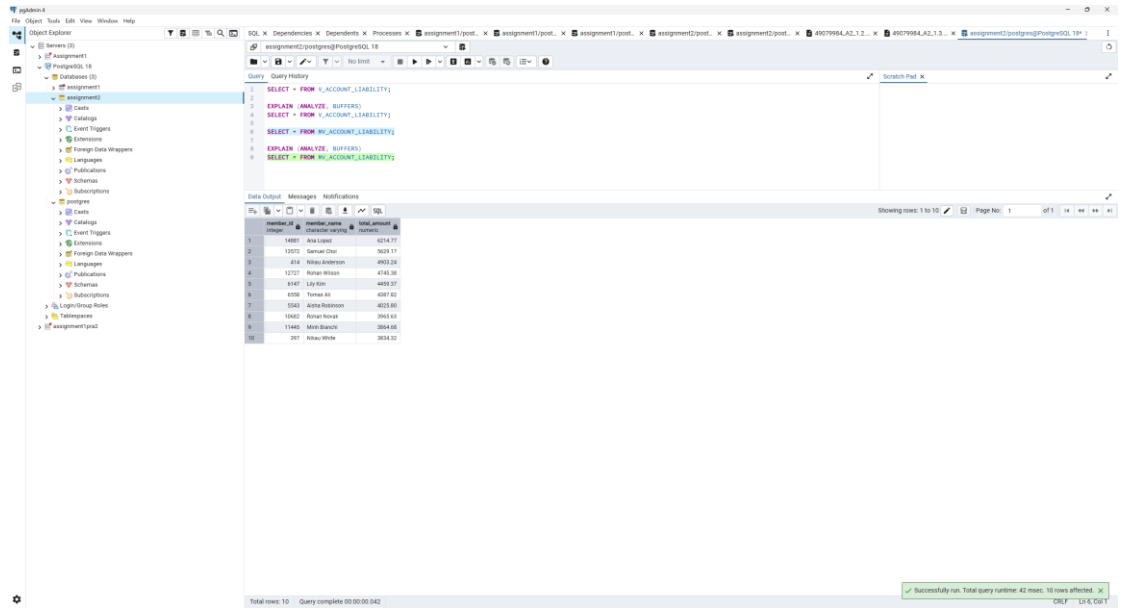
Q1.3.2 statement 2 | 2 of 2

```
      Batches: 1  Memory Usage: 4625kB
      Buffers: shared hit=2470
      ->  Hash Join  (cost=6265.10..252907.78 rows=3152614 width=22) (actual
        time=21.371..47.198 rows=30275.00 loops=1)
        Hash Cond: (t.member_id = e.member_id)
        Buffers: shared hit=2470
        ->  Hash Join  (cost=818.00..43094.59 rows=1868540 width=20) (actual
          time=3.584..21.249 rows=24000.00 loops=1)
          Hash Cond: (t.account_id = root.member_id)
          Buffers: shared hit=834
          ->  CTE Scan on account_tree t  (cost=0.00..37370.80 rows=1868540
            width=8) (actual time=0.025..12.580 rows=24000.00 loops=1)
            Storage: Memory  Maximum Storage: 1007kB
            Buffers: shared hit=556
          ->  Hash  (cost=518.00..518.00 rows=24000 width=16) (actual
            time=3.473..3.473 rows=24000.00 loops=1)
            Buckets: 32768  Batches: 1  Memory Usage: 1400kB
            Buffers: shared hit=278
            ->  Seq Scan on members root  (cost=0.00..518.00 rows=24000
              width=16) (actual time=0.003..1.903 rows=24000.00 loops=1)
              Buffers: shared hit=278
        ->  Hash  (cost=5073.70..5073.70 rows=29872 width=10) (actual
          time=17.720..17.724 rows=30275.00 loops=1)
          Buckets: 32768  Batches: 1  Memory Usage: 1557kB
          Buffers: shared hit=1636
          ->  Seq Scan on booking_events e  (cost=0.00..5073.70 rows=29872
            width=10) (actual time=7.108..15.167 rows=30275.00 loops=1)
            Filter: (((event_type)::text = ANY ('{Return,Incident}'::text[]))
              AND (event_time >= '2026-01-01 00:00:00'::timestamp without time
              zone) AND (event_time < '2026-07-01 00:00:00'::timestamp without
              time zone))
            Rows Removed by Filter: 166165
            Buffers: shared hit=1636
Planning:
 Buffers: shared hit=374
Planning Time: 3.853 ms
Execution Time: 61.402 ms
(52 rows)
```

Clean local replay | Actual psql output in a transcript viewer
Only whitespace and permitted long-result truncation changed

Screenshot Q1.3 (2)	Marks: /0.5

Query B original result



Query B materialized view, native pgAdmin execution plan

QUERY PLAN text
Seq Scan on mv_account_liability (cost=0.00..18.50 rows=850 width=68) (actual time=0.223..0.224 rows=10.00 loops=1)
Buffers: shared hit=1
Planning:
Buffers: shared hit=16
Planning Time: 1.373 ms
Execution Time: 0.236 ms

Native plan transcription

```
Seq Scan on mv_account_liability
  (cost=0.00..18.50 rows=850 width=68)
  (actual time=0.223..0.224 rows=10.00 loops=1)
  Buffers: shared hit=1
Planning:
  Buffers: shared hit=16
Planning Time: 1.373 ms
Execution Time: 0.236 ms
```

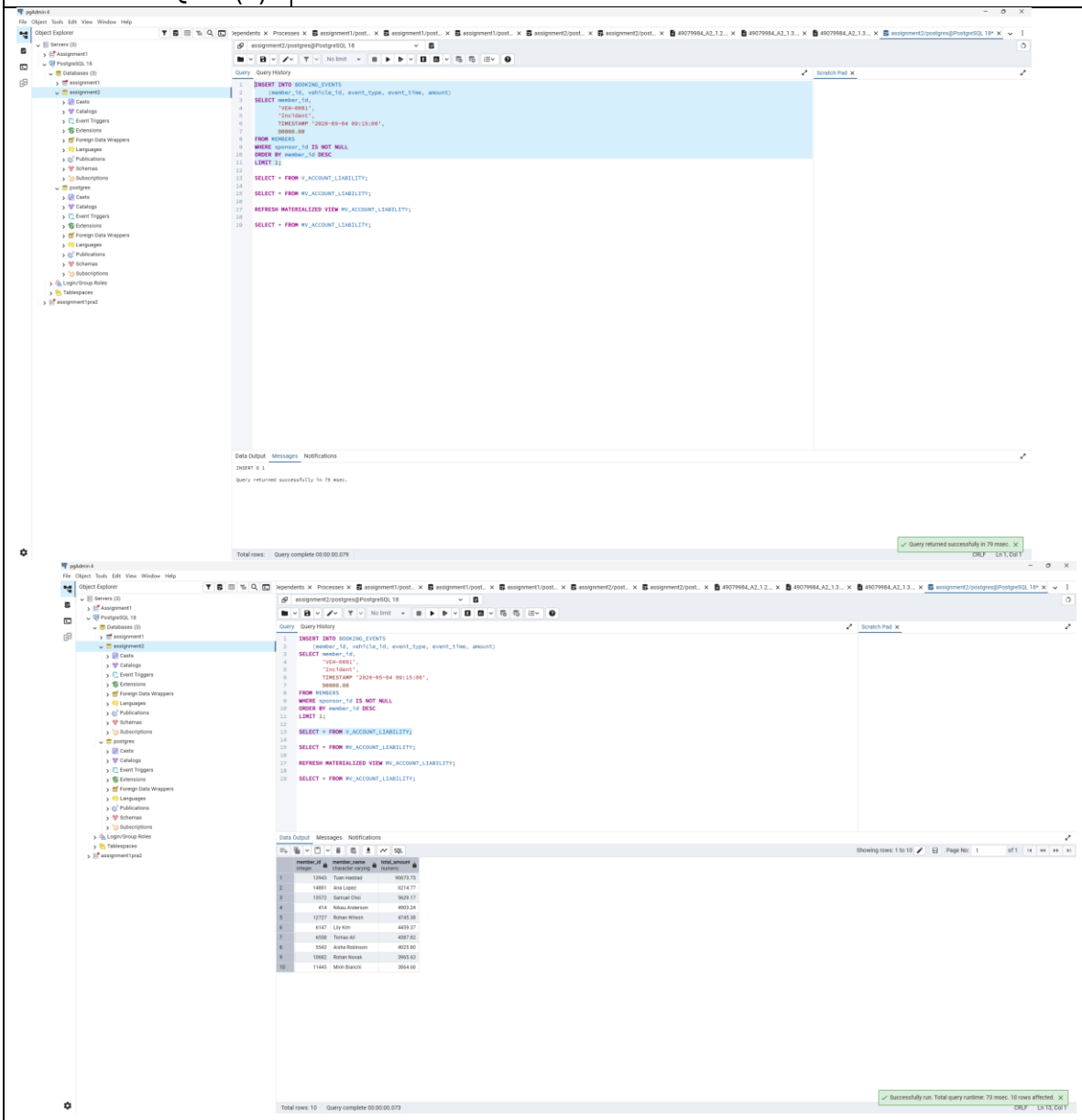
Explanation

Both original results initially show the same ten accounts, led by Ana Lopez (14881), 6,214.77. The new Query A plan builds the 24,000-member account mapping, filters 30,276 eligible events using Bitmap Index Scan and Bitmap Heap Scan, then performs joins, HashAggregate and a top-N sort. Its top cost is 370929.67..370929.69, planning takes 0.384 ms and execution takes 45.543 ms. This later run is distinct from the initial results and supplementary replay.

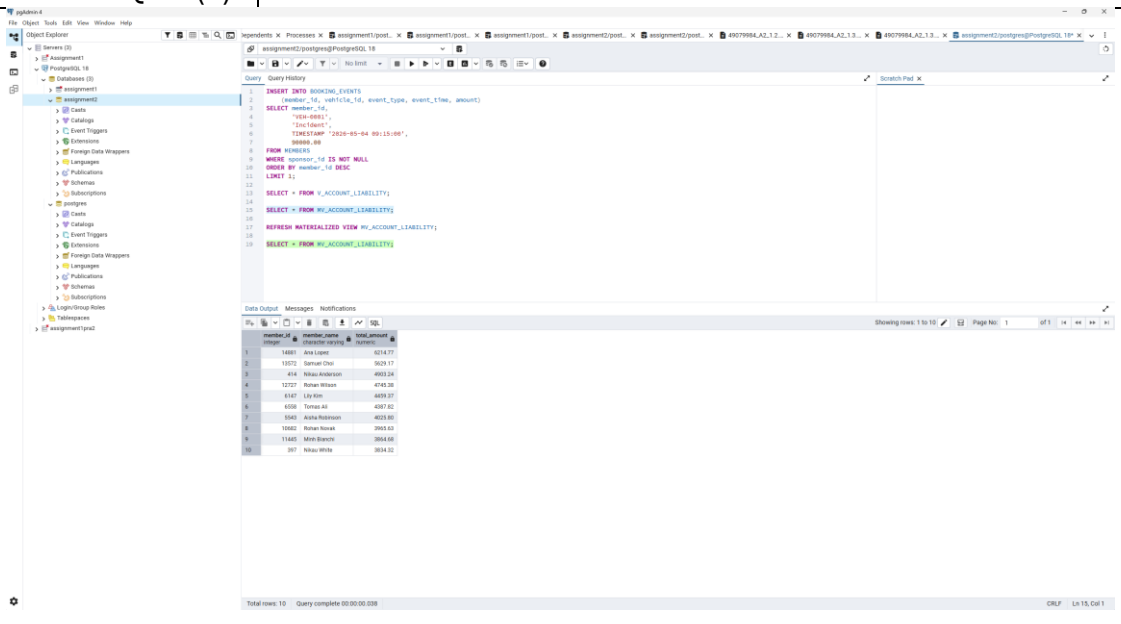
Query B scans the stored ten-row result: cost 0.00..18.50, planning 1.373 ms, execution 0.236 ms and one shared-buffer hit. Its estimate of 850 rows differs from the actual 10; it does not mean 850 rows are stored. Materialization avoids repeating recursion, joins and aggregation. Separate runs and database stages do not establish a controlled speed ratio. Use an explicit ORDER BY when output order matters.

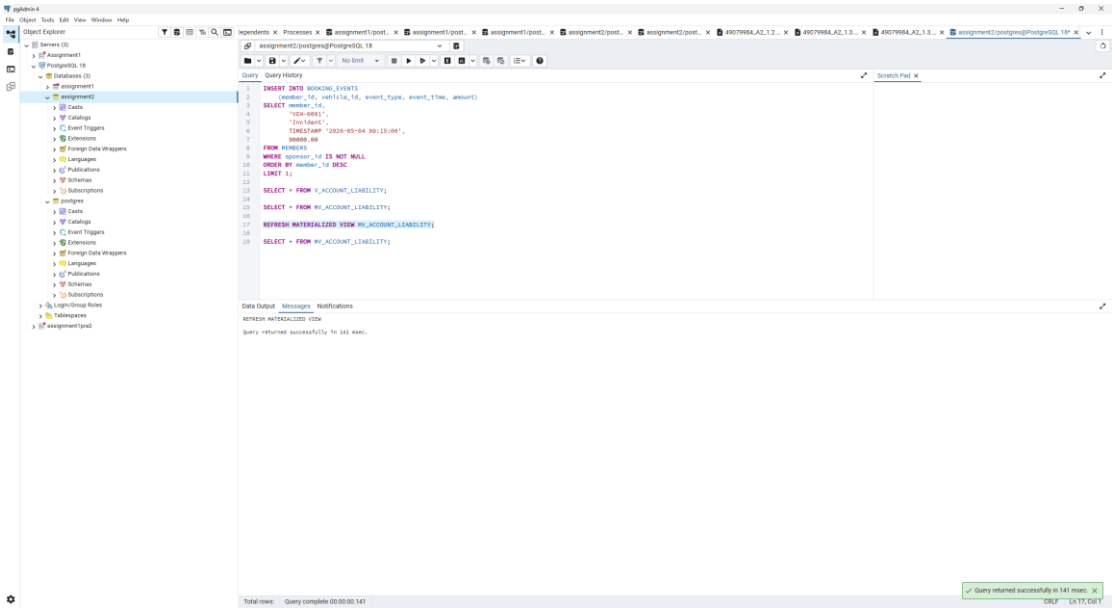
Comments:

Screenshot Q1.3 (3)	Marks: /0.5

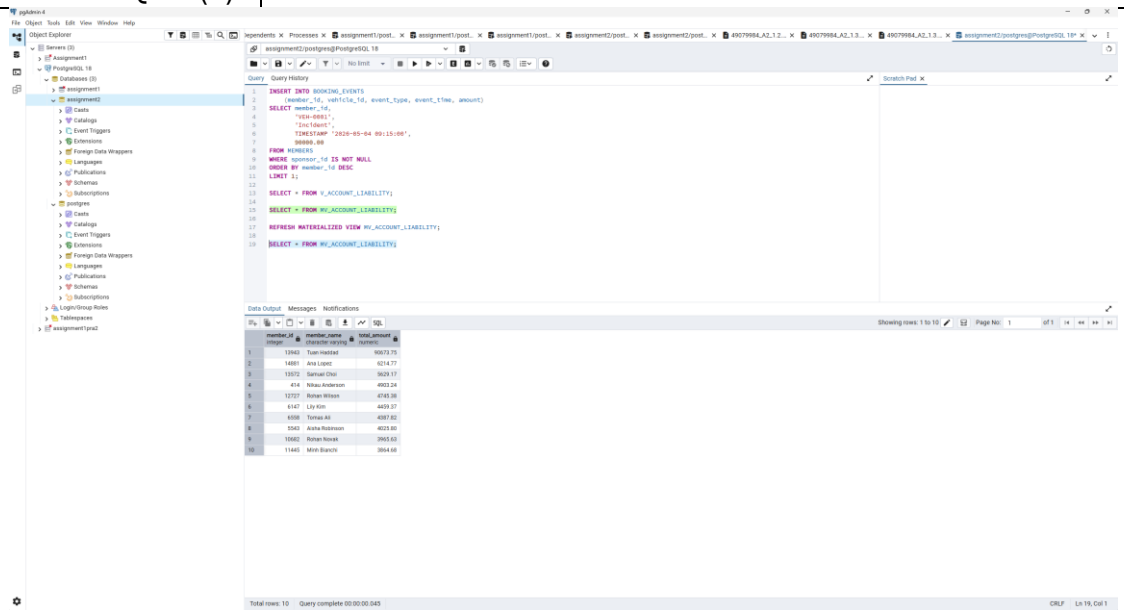


Screenshot Q1.3 (3)	Marks: /0.5





Screenshot Q1.3 (3)	Marks: /0.5



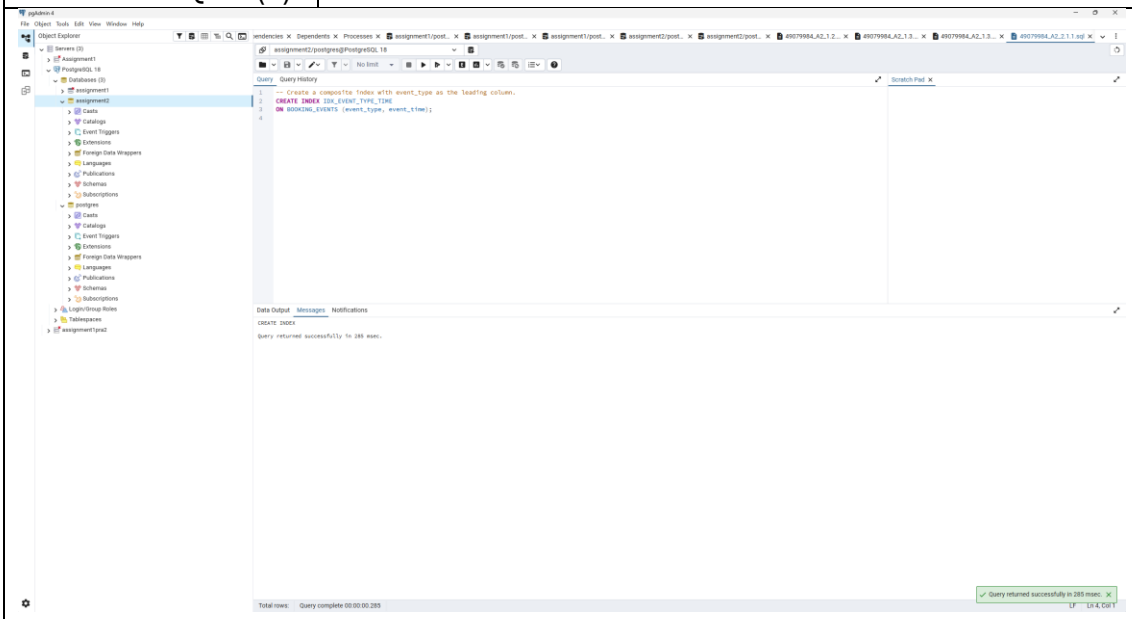
Explanation

The supplied INSERT chooses member 24000. Its sponsorship chain is 24000 -> 22086 -> 18148 -> 13943, so the additional 90,000.00 is billed to Tuan Haddad (13943). Query A immediately puts that account first at 90,673.75. Query B still shows the earlier top ten, led by Ana Lopez, and does not include Tuan Haddad. After REFRESH, Query B shows the same updated ten rows as Query A; account 397 drops out of the top ten.

A materialized view trades faster reads for extra storage, refresh work and potentially stale answers between refreshes. An ordinary view computes an answer from the data visible to each query.

Comments:

Screenshot Q2.1 (1)	Marks: /0.5



Explanation

This B-tree index has event_type as its first key and event_time as its second key, in the required order. It orders timestamps within each event type.

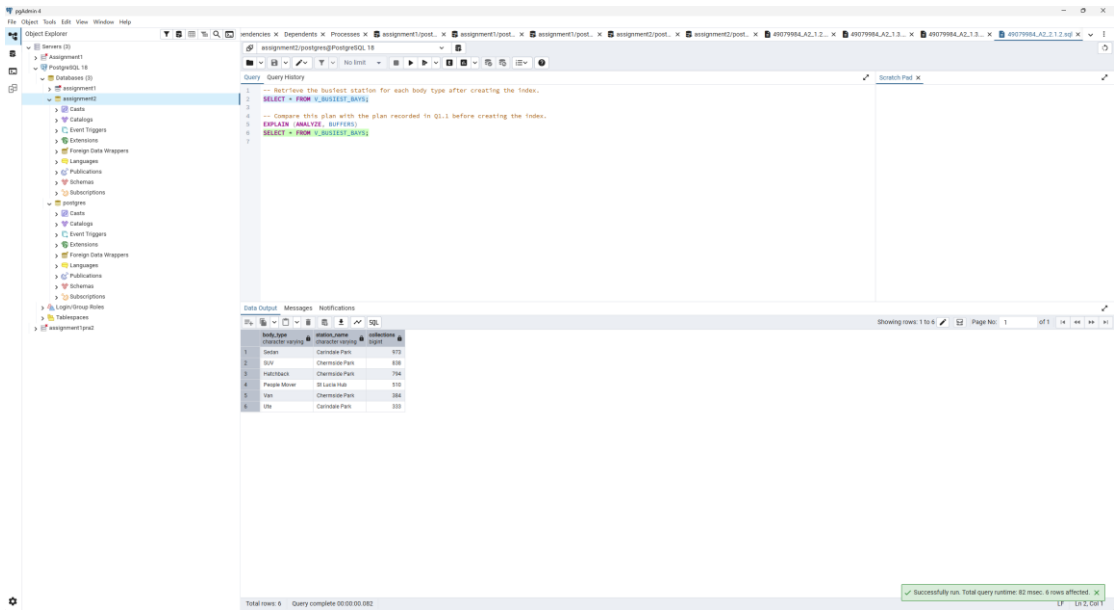
Comments:

Screenshot Q2.1 (2)	Marks: /1.5

SQL

```
-- Retrieve the busiest station for each body type after creating the index.
SELECT * FROM V_BUSIEST_BAYS;

-- Compare this plan with the plan recorded in Q1.1 before creating the index.
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM V_BUSIEST_BAYS;
```



Screenshot Q2.1 (2)	Marks: /1.5

Execution plan after the index-restoration step, native pgAdmin, lines 1 to 34

	QUERY PLAN text
1	Sort (cost=4786.08..4786.08 rows=2 width=32) (actual time=37.491..40.432 rows=6.00 loops=1)
2	Sort Key: ranked_stations.collections DESC, ranked_stations.body_type
3	Sort Method: quicksort Memory: 25kB
4	Buffers: shared hit=1664
5	-> Subquery Scan on ranked_stations (cost=4686.49..4786.07 rows=2 width=32) (actual time=37.303..40.427 rows=6.00 loops=1)
6	Filter: (ranked_stations.rn = 1)
7	Buffers: shared hit=1664
8	-> WindowAgg (cost=4686.49..4781.57 rows=360 width=47) (actual time=37.302..40.425 rows=6.00 loops=1)
9	Window: w1 AS (PARTITION BY collection_counts.body_type ORDER BY collection_counts.collections, collection_counts.station_id ROWS UNBOUNDED PRECEDING)
10	Run Condition: (row_number() OVER w1 <= 1)
11	Storage: Memory Maximum Storage: 17kB
12	Buffers: shared hit=1664
13	-> Incremental Sort (cost=4686.23..4774.37 rows=360 width=39) (actual time=37.292..40.403 rows=279.00 loops=1)
14	Sort Key: collection_counts.body_type, collection_counts.collections DESC, collection_counts.station_id
15	Presorted Key: collection_counts.body_type
16	Full-sort Groups: 6 Sort Method: quicksort Average Memory: 28kB Peak Memory: 28kB
17	Buffers: shared hit=1664
18	-> Subquery Scan on collection_counts (cost=4685.83..4763.77 rows=360 width=39) (actual time=37.245..40.319 rows=279.00 loops=1)
19	Buffers: shared hit=1664
20	-> Finalize GroupAggregate (cost=4685.83..4763.77 rows=360 width=39) (actual time=37.244..40.299 rows=279.00 loops=1)
21	Group Key: m.body_type, s.station_id
22	Buffers: shared hit=1664
23	-> Gather Merge (cost=4685.83..4755.58 rows=612 width=39) (actual time=37.234..40.251 rows=330.00 loops=1)
24	Workers Planned: 1
25	Workers Launched: 1
26	Buffers: shared hit=1664
27	-> Sort (cost=3685.82..3686.72 rows=360 width=39) (actual time=19.089..19.096 rows=165.00 loops=2)
28	Sort Key: m.body_type, s.station_id
29	Sort Method: quicksort Memory: 41kB
30	Buffers: shared hit=1664
31	Worker 0: Sort Method: quicksort Memory: 27kB
32	-> Partial HashAggregate (cost=3666.93..3670.53 rows=360 width=39) (actual time=18.906..18.918 rows=165.00 loops=2)
33	Group Key: m.body_type, s.station_id
34	Batches: 1 Memory Usage: 72kB

Execution plan after the index-restoration step, native pgAdmin, lines 35 to 68

	QUERY PLAN text
35	Buffers: shared hit=1656
36	Worker 0: Batches: 1 Memory Usage: 48kB
37	-> Hash Join (cost=32.62..3404.07 rows=35048 width=31) (actual time=0.698..15.107 rows=29769.50 loops=2)
38	Hash Cond: ((v.station_id)::text = (s.station_id)::text)
39	Buffers: shared hit=1656
40	-> Hash Join (cost=30.27..3303.22 rows=35048 width=14) (actual time=0.464..11.802 rows=29769.50 loops=2)
41	Hash Cond: ((v.model_code)::text = (m.model_code)::text)
42	Buffers: shared hit=1654
43	-> Hash Join (cost=27.25..3203.87 rows=35048 width=15) (actual time=0.260..8.568 rows=29769.50 loops=2)
44	Hash Cond: ((e.vehicle_id)::text = (v.vehicle_id)::text)
45	Buffers: shared hit=1652
46	-> Parallel Seq Scan on booking_events e (cost=0.00..3084.18 rows=35048 width=9) (actual time=0.006..4.972 rows=29769.50 loops=2)
47	Filter: ((event_type)::text = 'Collect'::text)
48	Rows Removed by Filter: 68453
49	Buffers: shared hit=1638
50	-> Hash (cost=16.00..16.00 rows=900 width=24) (actual time=0.249..0.249 rows=900.00 loops=2)
51	Buckets: 1024 Batches: 1 Memory Usage: 58kB
52	Buffers: shared hit=14
53	-> Seq Scan on vehicles v (cost=0.00..16.00 rows=900 width=24) (actual time=0.140..0.188 rows=900.00 loops=2)
54	Buffers: shared hit=14
55	-> Hash (cost=1.90..1.90 rows=90 width=15) (actual time=0.198..0.198 rows=90.00 loops=2)
56	Buckets: 1024 Batches: 1 Memory Usage: 13kB
57	Buffers: shared hit=2
58	-> Seq Scan on models m (cost=0.00..1.90 rows=90 width=15) (actual time=0.186..0.190 rows=90.00 loops=2)
59	Buffers: shared hit=2
60	-> Hash (cost=1.60..1.60 rows=60 width=24) (actual time=0.228..0.229 rows=60.00 loops=2)
61	Buckets: 1024 Batches: 1 Memory Usage: 12kB
62	Buffers: shared hit=2
63	-> Seq Scan on stations s (cost=0.00..1.60 rows=60 width=24) (actual time=0.215..0.218 rows=60.00 loops=2)
64	Buffers: shared hit=2
65	Planning:
66	Buffers: shared hit=54
67	Planning Time: 1.633 ms
68	Execution Time: 40.538 ms

Screenshot Q2.1 (2)	Marks: /1.5

Explanation

The six result rows are unchanged. The native plan remains a Parallel Seq Scan of the event log followed by hash joins, aggregation, ranking and sorting; IDX_EVENT_TYPE_TIME is not used by this query. Its top cost is 4786.08..4786.08, the same as the native plan without the index. Planning takes 1.633 ms and execution takes 40.538 ms, compared with 7.797 ms and 52.686 ms respectively in Q1.1.2. A faster single run with an unchanged access path can reflect caching and run-to-run variation; it is not evidence of an index benefit.

The scan returns 59,539 Collect events distributed throughout the heap. The query also needs vehicle_id, which this index does not contain. An index path would traverse many index entries and fetch many heap pages, whereas a parallel sequential scan reads the table efficiently. There is no timestamp condition, so the second index column does not narrow this query. Adding an index gives the planner another option; it does not require its use.

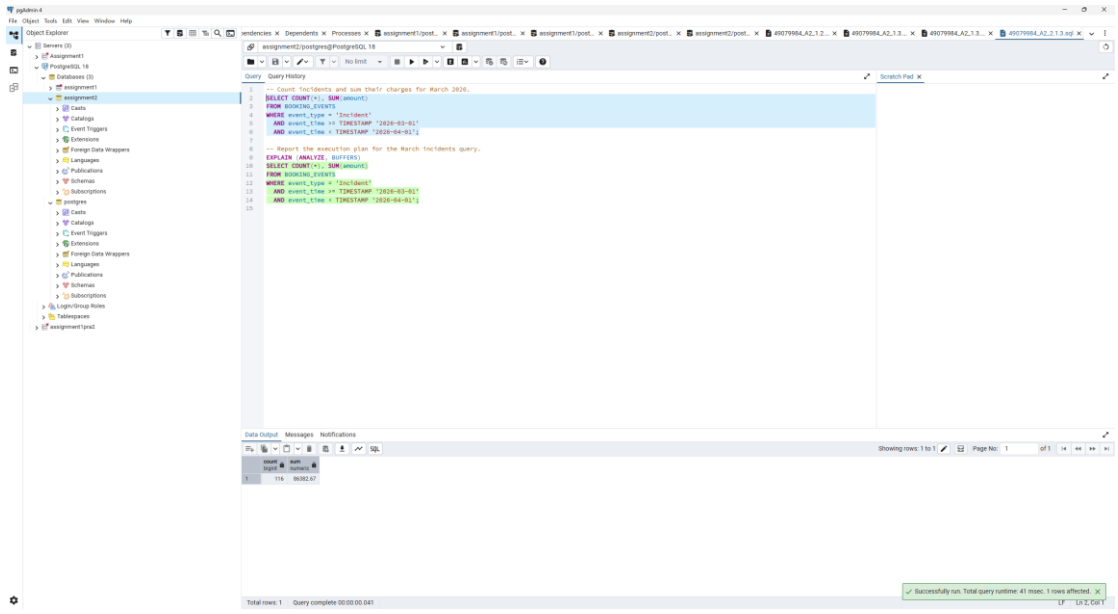
Comments:

Screenshot Q2.1 (3)	Marks: /1

SQL

```
-- Count incidents and sum their charges for March 2026.
SELECT COUNT(*), SUM(amount)
FROM BOOKING_EVENTS
WHERE event_type = 'Incident'
  AND event_time >= TIMESTAMP '2026-03-01'
  AND event_time < TIMESTAMP '2026-04-01';

-- Report the execution plan for the March incidents query.
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*), SUM(amount)
FROM BOOKING_EVENTS
WHERE event_type = 'Incident'
  AND event_time >= TIMESTAMP '2026-03-01'
  AND event_time < TIMESTAMP '2026-04-01';
```



Screenshot Q2.1 (3)	Marks: /1

Q2.1.3 March incidents plan — complete replay output

Q2.1.3 statement 2 | 1 of 1

```
QUERY PLAN
------------------------------------------------------------------------
Aggregate  (cost=388.48..388.49 rows=1 width=40) (actual time=0.315..0.316
  rows=1.00 loops=1)
 Buffers: shared hit=82
 ->  Bitmap Heap Scan on booking_events  (cost=5.99..387.86 rows=123 width=6)
   (actual time=0.059..0.261 rows=116.00 loops=1)
   Recheck Cond: (((event_type)::text = 'Incident'::text) AND (event_time >=
     '2026-03-01 00:00:00'::timestamp without time zone) AND (event_time <
     '2026-04-01 00:00:00'::timestamp without time zone))
   Heap Blocks: exact=79
   Buffers: shared hit=82
   ->  Bitmap Index Scan on idx_event_type_time  (cost=0.00..5.96 rows=123
     width=0) (actual time=0.034..0.035 rows=116.00 loops=1)
     Index Cond: (((event_type)::text = 'Incident'::text) AND (event_time >=
       '2026-03-01 00:00:00'::timestamp without time zone) AND (event_time <
       '2026-04-01 00:00:00'::timestamp without time zone))
     Index Searches: 1
     Buffers: shared hit=3
Planning:
 Buffers: shared hit=126
Planning Time: 2.155 ms
Execution Time: 0.390 ms
(14 rows)
```

Clean local replay | Actual psql output in a transcript viewer
Only whitespace and permitted long-result truncation changed

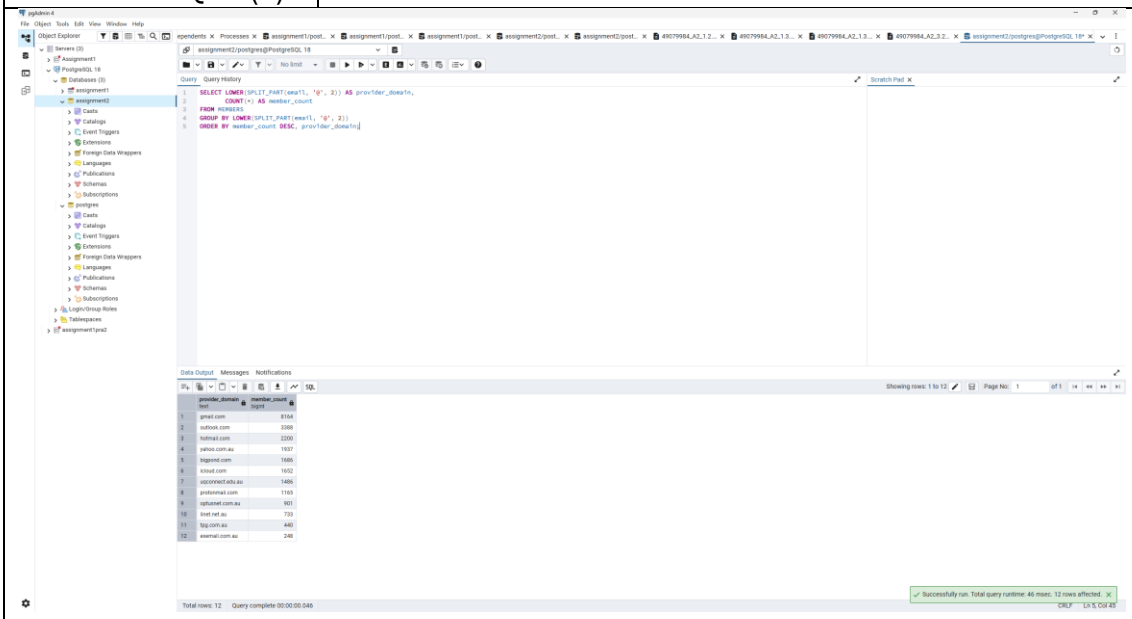
Explanation

The query returns count 116 and sum 86,382.67. The aggregate estimates and returns one result row, with cost 388.48..388.49 and execution time 0.390 ms. Its child Bitmap Index Scan finds 116 matching entries, versus 123 estimated, and Bitmap Heap Scan visits 79 exact heap blocks before computing the aggregate.

Incident equality fixes the leading key, and the lower and upper March timestamp bounds restrict the second key to a short contiguous range within Incident entries. These 116 events are about 0.059 percent of the table, so the bitmap index path avoids most rows and pages. The plan needs heap access for amount, which is not stored in the index. This is much more selective than the all-time Collect query in Q2.1 part 2.

Comments:

Screenshot Q2.2(1)	Marks: /1



Explanation

LOWER(SPLIT_PART(email, '@', 2)) extracts the part after @ and normalizes its case before grouping. The query reports 12 domains accounting for all 24,000 members, with gmail.com first at 8,164 members. exemail.com.au has 248 members. The secondary alphabetical ordering makes equal counts reproducible.

Comments:

Screenshot Q2.2 (2)	Marks: /1

SQL

```
-- Report the domain-count query plan before creating the expression index.
EXPLAIN (ANALYZE, BUFFERS)
SELECT LOWER(SPLIT_PART(email, '@', 2)) AS provider_domain,
       COUNT(*) AS member_count
FROM MEMBERS
GROUP BY LOWER(SPLIT_PART(email, '@', 2))
ORDER BY member_count DESC, provider_domain;
```

Q2.2.2 domain counts before indexing — complete replay output

Q2.2.2 statement 1 | 1 of 1

```
QUERY PLAN
------------------------------------------------------------------------
Sort  (cost=2864.09..2924.09 rows=24000 width=40) (actual time=6.749..6.751
  rows=12.00 loops=1)
 Sort Key: (count(*)) DESC, (lower(split_part((email)::text, '@'::text, 2)))
 Sort Method: quicksort  Memory: 25kB
 Buffers: shared hit=284
 ->  HashAggregate  (cost=758.00..1118.00 rows=24000 width=40) (actual
   time=6.683..6.720 rows=12.00 loops=1)
   Group Key: lower(split_part((email)::text, '@'::text, 2))
   Batches: 1  Memory Usage: 545kB
   Buffers: shared hit=278
   ->  Seq Scan on members  (cost=0.00..638.00 rows=24000 width=32) (actual
     time=0.023..4.513 rows=24000.00 loops=1)
     Buffers: shared hit=278
Planning:
 Buffers: shared hit=139
Planning Time: 1.444 ms
Execution Time: 6.997 ms
(14 rows)
```

Clean local replay | Actual psql output in a transcript viewer
Only whitespace and permitted long-result truncation changed

Explanation

The plan performs a Seq Scan of all 24,000 MEMBERS rows, computes the normalized domain, uses HashAggregate for the counts, and sorts the resulting 12 groups by count descending. The top cost is 2864.09..2924.09 and execution time is 6.997 ms. It estimates 24,000 groups but actually returns only 12, reflecting the lack of statistics for this derived expression at this point.

Every member contributes to the requested whole-population counts, so a complete scan is appropriate. Hash aggregation has a single batch and the final quicksort uses only 25 kB; neither step spills to disk in this run.

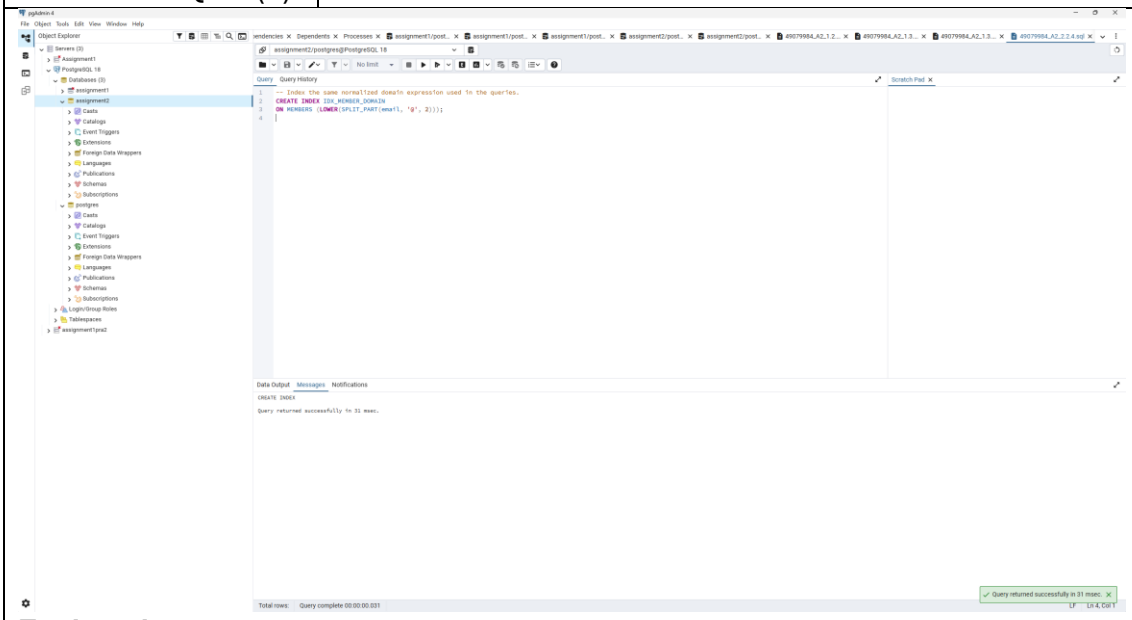
Comments:

Screenshot Q2.2 (3)	Marks: /1

The UNIQUE index on email orders complete email addresses, so members with the same provider domain are not necessarily adjacent. It does not index the normalized domain expression, so it cannot directly locate a domain or avoid processing every address for the domain-count query.

Screenshot Q2.2 (3)	Marks: /1
Comments:	

Screenshot Q2.2 (4)	Marks: /0.5



Explanation

The expression in IDX_MEMBER_DOMAIN exactly matches LOWER(SPLIT_PART(email, '@', 2)) in the queries, including the case folding. PostgreSQL can therefore look up a normalized domain directly in this index.

Comments:

Screenshot Q2.2 (5)	Marks: /1.5

SQL

```
-- Report the domain-count query plan after creating the expression index.
EXPLAIN (ANALYZE, BUFFERS)
SELECT LOWER(SPLIT_PART(email, '@', 2)) AS provider_domain,
       COUNT(*) AS member_count
FROM MEMBERS
GROUP BY LOWER(SPLIT_PART(email, '@', 2))
ORDER BY member_count DESC, provider_domain;

-- Report the selective lookup plan for a rare provider domain.
EXPLAIN (ANALYZE, BUFFERS)
SELECT member_id, member_name, email
FROM MEMBERS
WHERE LOWER(SPLIT_PART(email, '@', 2)) = 'exemail.com.au';
```

Q2.2.5 domain counts after indexing — complete replay output

Q2.2.5 statement 1 | 1 of 1

```
QUERY PLAN
------------------------------------------------------------------------
Sort  (cost=2864.09..2924.09 rows=24000 width=40) (actual time=5.016..5.019
  rows=12.00 loops=1)
 Sort Key: (count(*)) DESC, (lower(split_part((email)::text, '@'::text, 2)))
 Sort Method: quicksort  Memory: 25kB
 Buffers: shared hit=284
 ->  HashAggregate  (cost=758.00..1118.00 rows=24000 width=40) (actual
   time=4.936..4.984 rows=12.00 loops=1)
   Group Key: lower(split_part((email)::text, '@'::text, 2))
   Batches: 1  Memory Usage: 545kB
   Buffers: shared hit=278
   ->  Seq Scan on members  (cost=0.00..638.00 rows=24000 width=32) (actual
     time=0.024..3.325 rows=24000.00 loops=1)
     Buffers: shared hit=278
Planning:
 Buffers: shared hit=153 read=1
Planning Time: 2.519 ms
Execution Time: 5.274 ms
(14 rows)
```

Clean local replay | Actual psql output in a transcript viewer
Only whitespace and permitted long-result truncation changed

Screenshot Q2.2 (5)	Marks: /1.5

Q2.2.5 rare provider lookup — complete replay output

Q2.2.5 statement 2 | 1 of 1

```
QUERY PLAN
------------------------------------------------------------------------
Bitmap Heap Scan on members  (cost=5.22..226.08 rows=120 width=45) (actual
  time=0.079..0.431 rows=248.00 loops=1)
 Recheck Cond: (lower(split_part((email)::text, '@'::text, 2)) =
   'exemail.com.au'::text)
 Heap Blocks: exact=164
 Buffers: shared hit=164 read=2
 ->  Bitmap Index Scan on idx_member_domain  (cost=0.00..5.19 rows=120
   width=0) (actual time=0.057..0.057 rows=248.00 loops=1)
   Index Cond: (lower(split_part((email)::text, '@'::text, 2)) =
     'exemail.com.au'::text)
   Index Searches: 1
   Buffers: shared read=2
Planning:
 Buffers: shared hit=125
Planning Time: 1.807 ms
Execution Time: 0.510 ms
(12 rows)
```

Clean local replay | Actual psql output in a transcript viewer
Only whitespace and permitted long-result truncation changed

Explanation

The domain-count query keeps its Seq Scan -> HashAggregate -> Sort plan, estimates 24,000 groups and returns 12. Its top cost stays 2864.09..2924.09; execution is 5.274 ms compared with 6.997 ms before the index. All 24,000 members still have to contribute to the counts, and an index ordered by domain cannot directly satisfy ORDER BY COUNT(*) DESC.

The rare-provider lookup instead uses Bitmap Index Scan on IDX_MEMBER_DOMAIN followed by Bitmap Heap Scan. It returns 248 members, about 1.03 percent of MEMBERS, versus 120 estimated, with cost 5.22..226.08 and execution time 0.510 ms. The bitmap batches access to 164 exact heap blocks; heap access is needed for member_id, member_name and email. The index avoids evaluating the domain predicate on every member. Its advantage is selective access, rather than speeding every query that mentions the expression.

Comments:

Screenshot Q3.1 (1)	Marks: /1.5

SQL

```
-- Statement A: find vehicles with no Collect event using NOT EXISTS.
SELECT v.vehicle_id
FROM VEHICLES v
WHERE NOT EXISTS (
    SELECT 1
    FROM BOOKING_EVENTS e
    WHERE e.vehicle_id = v.vehicle_id
      AND e.event_type = 'Collect'
)
ORDER BY v.vehicle_id;

-- Statement B: find the same vehicles using a left join and a NULL check.
SELECT v.vehicle_id
FROM VEHICLES v
LEFT JOIN BOOKING_EVENTS e
    ON e.vehicle_id = v.vehicle_id
   AND e.event_type = 'Collect'
WHERE e.vehicle_id IS NULL
ORDER BY v.vehicle_id;
```

Statement A native pgAdmin result, first ten and last ten rows

Left: rows 1 to 10. Right: rows 66 to 75. The 55 intermediate rows are omitted.

	vehicle_id [PK] character varying
1	VEH-0019
2	VEH-0041
3	VEH-0045
4	VEH-0049
5	VEH-0057
6	VEH-0094
7	VEH-0098
8	VEH-0123
9	VEH-0137
10	VEH-0139

66	VEH-0820
67	VEH-0826
68	VEH-0845
69	VEH-0857
70	VEH-0860
71	VEH-0861
72	VEH-0874
73	VEH-0881
74	VEH-0882
75	VEH-0891

Total rows: 75

Screenshot Q3.1 (1)	Marks: /1.5

Statement B native pgAdmin result, first ten and last ten rows

Left: rows 1 to 10. Right: rows 66 to 75. The 55 intermediate rows are omitted.

	vehicle_id [PK] character varying		
1	VEH-0019	66	VEH-0820
2	VEH-0041	67	VEH-0826
3	VEH-0045	68	VEH-0845
4	VEH-0049	69	VEH-0857
5	VEH-0057	70	VEH-0860
6	VEH-0094	71	VEH-0861
7	VEH-0098	72	VEH-0874
8	VEH-0123	73	VEH-0881
9	VEH-0137	74	VEH-0882
10	VEH-0139	75	VEH-0891

Total rows: 75

Explanation

Both statements return the same 75 vehicle IDs, ordered by vehicle_id. Statement A uses a correlated NOT EXISTS test. Statement B places the Collect predicate in the LEFT JOIN condition and keeps rows with no matching event vehicle_id. The matched-side vehicle_id is NOT NULL in the schema, so NULL identifies an unmatched vehicle unambiguously.

Putting event_type in WHERE instead of ON would remove the unmatched rows and break the outer-join solution. Independent EXCEPT ALL checks in both directions previously found no differences. The native result captures show the same first ten and last ten IDs and a total of 75 rows. They are placed in the supplied Statement A then Statement B order alongside the SQL above; the captured result panes themselves do not display the SQL text.

Comments:

Screenshot Q3.1 (2)	Marks: /1.5

SQL

```
-- Report the execution plan for Statement A.
EXPLAIN (ANALYZE, BUFFERS)
SELECT v.vehicle_id
FROM VEHICLES v
WHERE NOT EXISTS (
    SELECT 1
    FROM BOOKING_EVENTS e
    WHERE e.vehicle_id = v.vehicle_id
      AND e.event_type = 'Collect'
)
ORDER BY v.vehicle_id;

-- Report the execution plan for Statement B.
EXPLAIN (ANALYZE, BUFFERS)
SELECT v.vehicle_id
FROM VEHICLES v
LEFT JOIN BOOKING_EVENTS e
    ON e.vehicle_id = v.vehicle_id
   AND e.event_type = 'Collect'
WHERE e.vehicle_id IS NULL
ORDER BY v.vehicle_id;
```

Screenshot Q3.1 (2)	Marks: /1.5

Statement A execution plan — complete replay output

Q3.1.2 statement 1 | 1 of 1

```
QUERY PLAN
------------------------------------------------------------------------
Sort  (cost=4576.31..4576.43 rows=49 width=9) (actual time=17.213..17.218
  rows=75.00 loops=1)
 Sort Key: v.vehicle_id
 Sort Method: quicksort  Memory: 25kB
 Buffers: shared hit=1877
 ->  Hash Right Anti Join  (cost=1412.86..4574.93 rows=49 width=9) (actual
   time=17.159..17.172 rows=75.00 loops=1)
   Hash Cond: ((e.vehicle_id)::text = (v.vehicle_id)::text)
   Buffers: shared hit=1874
   ->  Bitmap Heap Scan on booking_events e  (cost=1385.61..3765.48 rows=59509
     width=9) (actual time=1.519..11.846 rows=59539.00 loops=1)
     Recheck Cond: ((event_type)::text = 'Collect'::text)
     Heap Blocks: exact=1636
     Buffers: shared hit=1867
     ->  Bitmap Index Scan on idx_event_type_time  (cost=0.00..1370.74
       rows=59509 width=0) (actual time=1.418..1.419 rows=59539.00 loops=1)
       Index Cond: ((event_type)::text = 'Collect'::text)
       Index Searches: 1
       Buffers: shared hit=231
   ->  Hash  (cost=16.00..16.00 rows=900 width=9) (actual time=0.128..0.130
     rows=900.00 loops=1)
     Buckets: 1024  Batches: 1  Memory Usage: 45kB
     Buffers: shared hit=7
     ->  Seq Scan on vehicles v  (cost=0.00..16.00 rows=900 width=9) (actual
       time=0.013..0.063 rows=900.00 loops=1)
       Buffers: shared hit=7
Planning:
 Buffers: shared hit=233
Planning Time: 3.021 ms
Execution Time: 17.320 ms
(24 rows)
```

Clean local replay | Actual psql output in a transcript viewer
Only whitespace and permitted long-result truncation changed

Screenshot Q3.1 (2)	Marks: /1.5

Statement B execution plan — complete replay output

Q3.1.2 statement 2 | 1 of 1

```
QUERY PLAN
------------------------------------------------------------------------
Sort  (cost=4576.31..4576.43 rows=49 width=9) (actual time=14.521..14.525
  rows=75.00 loops=1)
 Sort Key: v.vehicle_id
 Sort Method: quicksort  Memory: 25kB
 Buffers: shared hit=1877
 ->  Hash Right Anti Join  (cost=1412.86..4574.93 rows=49 width=9) (actual
   time=14.462..14.473 rows=75.00 loops=1)
   Hash Cond: ((e.vehicle_id)::text = (v.vehicle_id)::text)
   Buffers: shared hit=1874
   ->  Bitmap Heap Scan on booking_events e  (cost=1385.61..3765.48 rows=59509
     width=9) (actual time=1.654..10.138 rows=59539.00 loops=1)
     Recheck Cond: ((event_type)::text = 'Collect'::text)
     Heap Blocks: exact=1636
     Buffers: shared hit=1867
     ->  Bitmap Index Scan on idx_event_type_time  (cost=0.00..1370.74
       rows=59509 width=0) (actual time=1.547..1.547 rows=59539.00 loops=1)
       Index Cond: ((event_type)::text = 'Collect'::text)
       Index Searches: 1
       Buffers: shared hit=231
   ->  Hash  (cost=16.00..16.00 rows=900 width=9) (actual time=0.120..0.121
     rows=900.00 loops=1)
     Buckets: 1024  Batches: 1  Memory Usage: 45kB
     Buffers: shared hit=7
     ->  Seq Scan on vehicles v  (cost=0.00..16.00 rows=900 width=9) (actual
       time=0.016..0.062 rows=900.00 loops=1)
       Buffers: shared hit=7
Planning:
 Buffers: shared hit=233
Planning Time: 2.650 ms
Execution Time: 14.630 ms
(24 rows)
```

Clean local replay | Actual psql output in a transcript viewer
Only whitespace and permitted long-result truncation changed

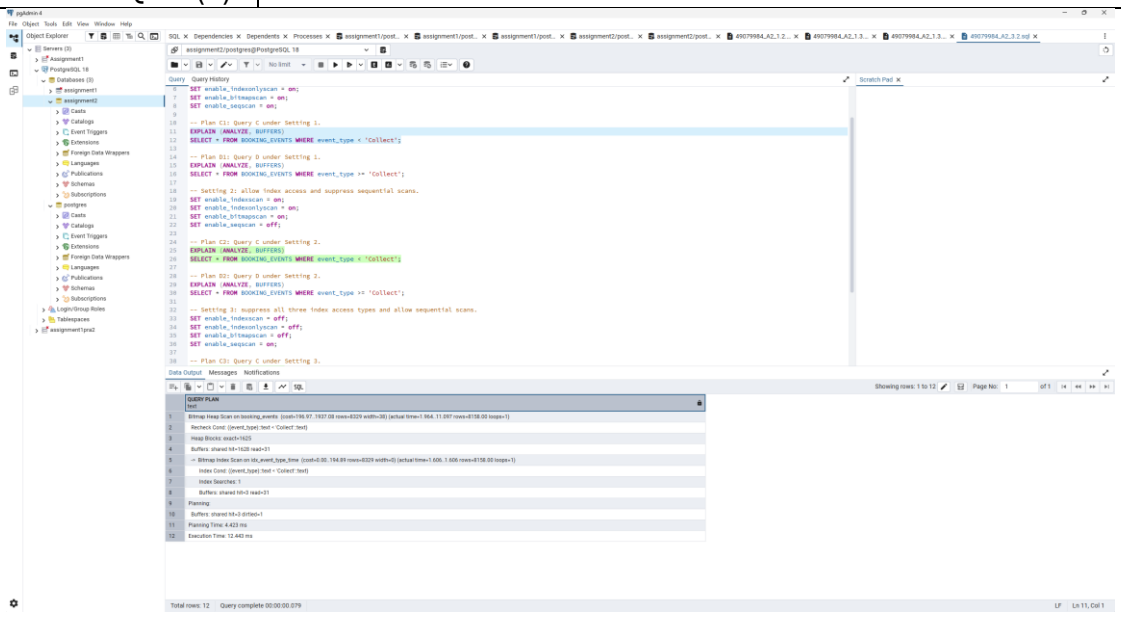
Explanation

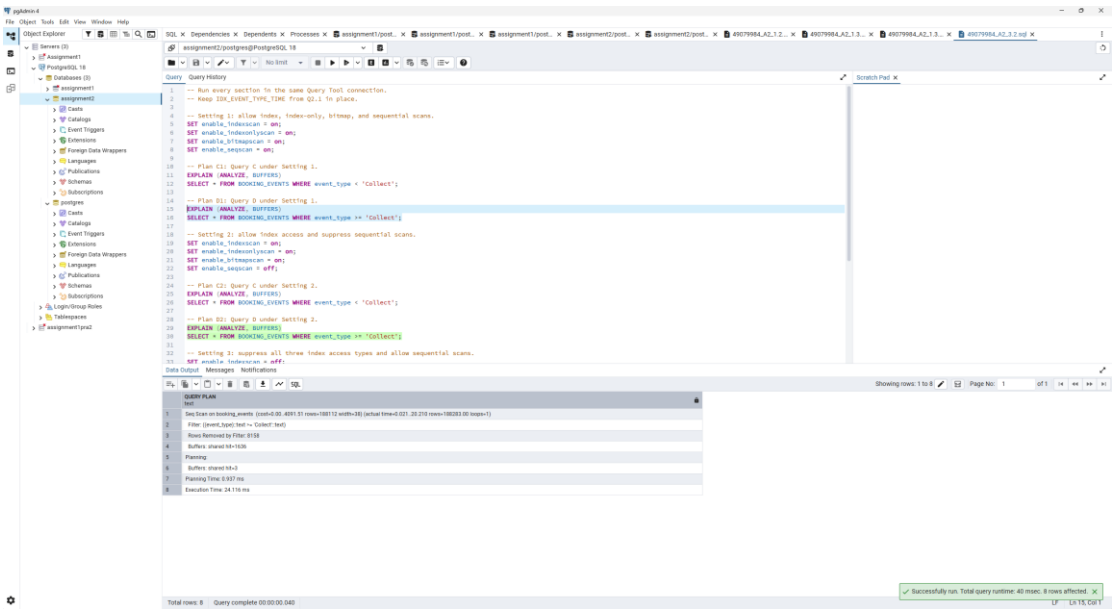
The two plans have the same operator structure and cost 4576.31..4576.43. Both return 75 rows, versus 49 estimated, using Hash Right Anti Join and a final vehicle_id sort. Statement A takes 17.320 ms and Statement B 14.630 ms in these runs.

PostgreSQL decorrelates NOT EXISTS and recognizes the outer-join NULL filter as the same anti-join operation. It hashes the small set of 900 vehicles and scans 59,539 Collect events through a bitmap index/heap path, keeping the vehicle rows without any match. The Right direction describes the physical join inputs chosen by the optimizer; it preserves the intended vehicle-side anti-join semantics. Neither syntax forces a separate full event scan for each vehicle. The minor time difference does not establish a consistent performance difference.

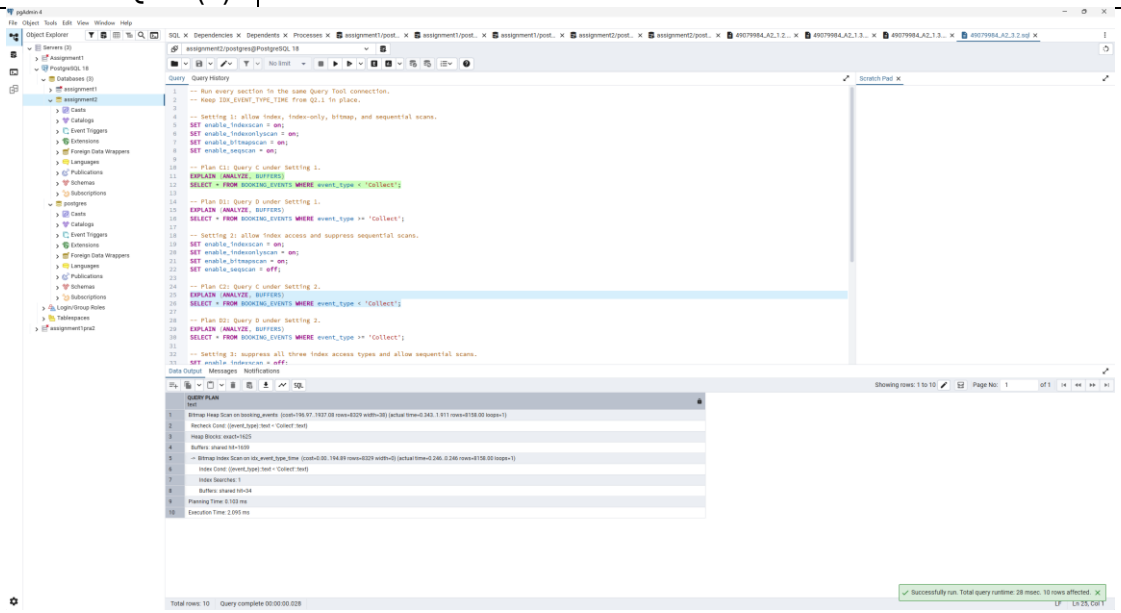
Comments:

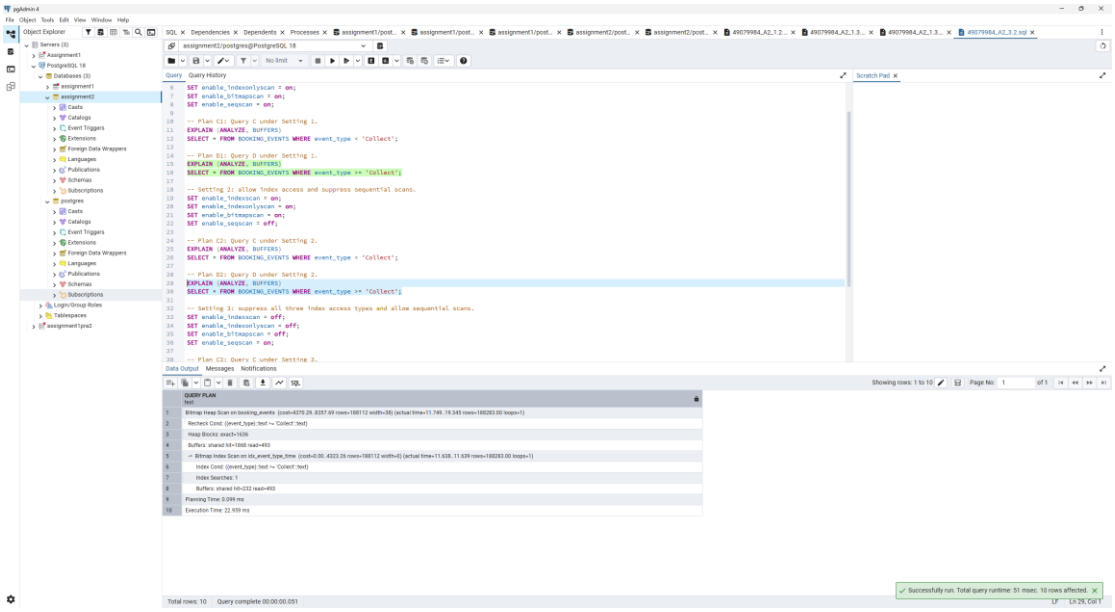
Screenshot Q3.2 (1)	Marks: /2



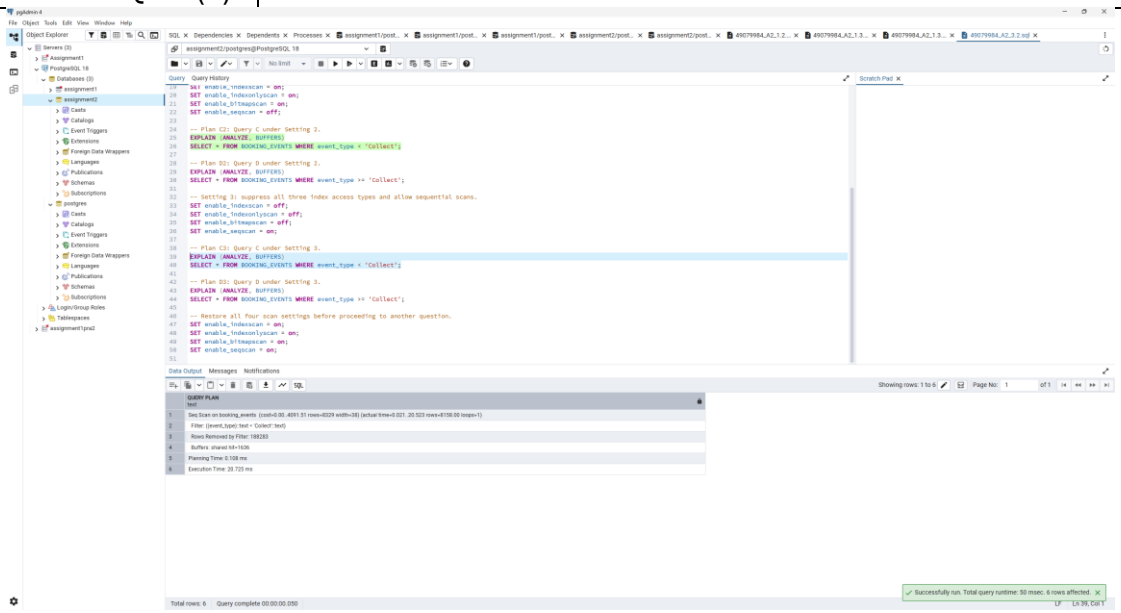


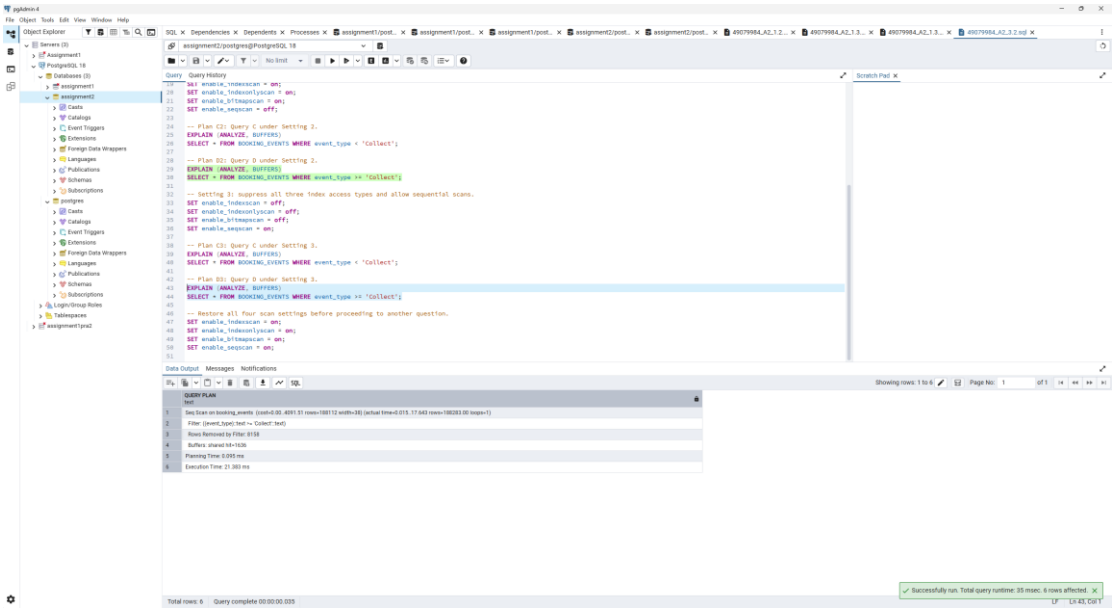
Screenshot Q3.2 (1)	Marks: /2



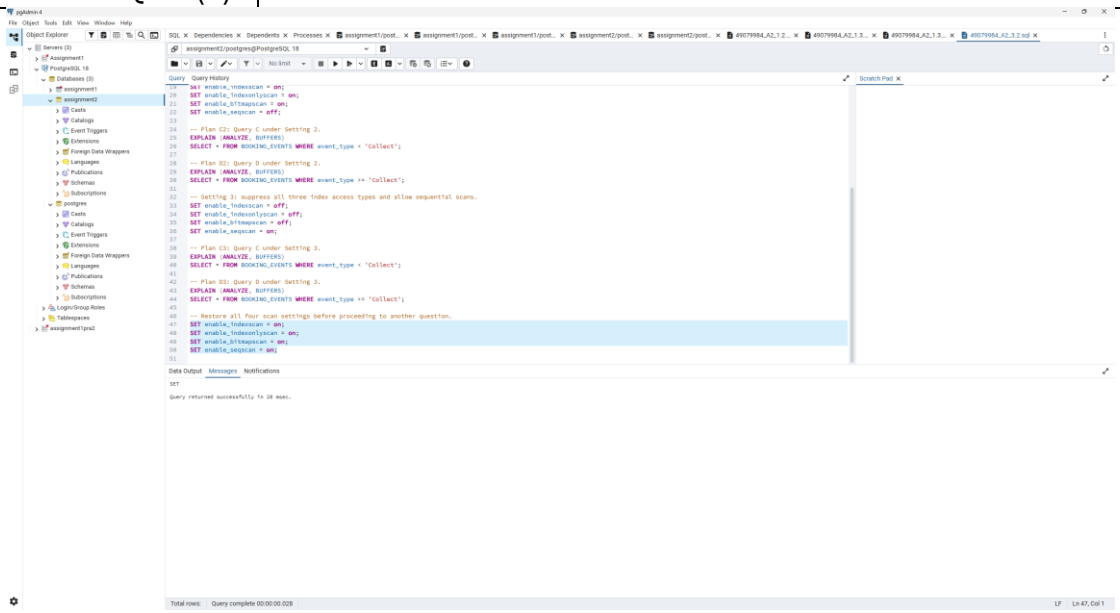


Screenshot Q3.2 (1)	Marks: /2





Screenshot Q3.2 (1)	Marks: /2



Screenshot Q3.2 (1)	Marks: /2

Full text of the recorded plans

These are transcriptions of the original interactive pgAdmin plans above, with unchanged values. They belong to the recorded run, rather than the clean replay used to supplement Q1–Q3.1.

C1

```
Bitmap Heap Scan on booking_events (cost=196.97..1937.08 rows=8329 width=38)
    (actual time=1.964..11.097 rows=8158.00 loops=1)
  Recheck Cond: ((event_type)::text < 'Collect'::text)
  Heap Blocks: exact=1625
  Buffers: shared hit=1628 read=31
  -> Bitmap Index Scan on idx_event_type_time (cost=0.00..194.89 rows=8329
    width=0) (actual time=1.606..1.606 rows=8158.00 loops=1)
     Index Cond: ((event_type)::text < 'Collect'::text)
     Index Searches: 1
     Buffers: shared hit=3 read=31
Planning:
  Buffers: shared hit=3 dirtied=1
Planning Time: 4.423 ms
Execution Time: 12.443 ms
```

D1

```
Seq Scan on booking_events (cost=0.00..4091.51 rows=188112 width=38) (actual
    time=0.021..20.210 rows=188283.00 loops=1)
  Filter: ((event_type)::text >= 'Collect'::text)
  Rows Removed by Filter: 8158
  Buffers: shared hit=1636
Planning:
  Buffers: shared hit=3
Planning Time: 0.937 ms
Execution Time: 24.116 ms
```

C2

```
Bitmap Heap Scan on booking_events (cost=196.97..1937.08 rows=8329 width=38)
    (actual time=0.343..1.911 rows=8158.00 loops=1)
  Recheck Cond: ((event_type)::text < 'Collect'::text)
  Heap Blocks: exact=1625
  Buffers: shared hit=1659
  -> Bitmap Index Scan on idx_event_type_time (cost=0.00..194.89 rows=8329
    width=0) (actual time=0.246..0.246 rows=8158.00 loops=1)
     Index Cond: ((event_type)::text < 'Collect'::text)
     Index Searches: 1
     Buffers: shared hit=34
Planning Time: 0.103 ms
Execution Time: 2.095 ms
```

D2

```
Bitmap Heap Scan on booking_events (cost=4370.29..8357.69 rows=188112
    width=38) (actual time=11.749..19.345 rows=188283.00 loops=1)
  Recheck Cond: ((event_type)::text >= 'Collect'::text)
  Heap Blocks: exact=1636
  Buffers: shared hit=1868 read=493
  -> Bitmap Index Scan on idx_event_type_time (cost=0.00..4323.26 rows=188112
    width=0) (actual time=11.638..11.639 rows=188283.00 loops=1)
     Index Cond: ((event_type)::text >= 'Collect'::text)
     Index Searches: 1
     Buffers: shared hit=232 read=493
Planning Time: 0.099 ms
Execution Time: 22.959 ms
```

Screenshot Q3.2 (1)	Marks: /2

C3

```
Seq Scan on booking_events (cost=0.00..4091.51 rows=8329 width=38) (actual
    time=0.021..20.523 rows=8158.00 loops=1)
  Filter: ((event_type)::text < 'Collect'::text)
  Rows Removed by Filter: 188283
  Buffers: shared hit=1636
Planning Time: 0.108 ms
Execution Time: 20.725 ms
```

D3

```
Seq Scan on booking_events (cost=0.00..4091.51 rows=188112 width=38) (actual
    time=0.015..17.643 rows=188283.00 loops=1)
  Filter: ((event_type)::text >= 'Collect'::text)
  Rows Removed by Filter: 8158
  Buffers: shared hit=1636
Planning Time: 0.095 ms
Execution Time: 21.383 ms
```

Comments:

Screenshot Q3.2 (2)	Marks: /2

Queries C and D partition 196,441 events at this stage. C returns 8,158 rows (4.15 percent), while D returns 188,283 (95.85 percent). The opposite inequalities on the same indexed column therefore have very different selectivity.

C1 uses Bitmap Index Scan on IDX_EVENT_TYPE_TIME followed by Bitmap Heap Scan. Its cost is 196. 97..1937.08, with 8,329 rows estimated and 8,158 returned. Planning takes 4.423 ms and execution 12.443 ms. It visits 1,625 exact heap blocks; total buffers are 1,628 shared hits and 31 reads. Selective index access costs less than scanning and filtering the whole event log, even though the matching events are spread across many heap pages.

D1 uses Seq Scan, with cost 0. 00..4091.51, 188,112 estimated rows and 188,283 actual rows. Planning takes 0.937 ms and execution 24.116 ms. It reads the heap through 1,636 shared buffer hits and removes 8,158 nonmatching rows. Returning almost the whole table makes a sequential pass cheaper in the cost model than traversing many index entries and then visiting almost every heap page.

C2 retains the bitmap plan and the same cost and row estimates when sequential scans are suppressed. Planning takes 0.103 ms and execution 2.095 ms. All 1,659 buffer accesses are hits, including 34 for the index. The change from C1 includes a warmer cache; suppressing a scan type did not change the chosen access method, so the faster time cannot be attributed to a new plan.

D2 uses Bitmap Index Scan plus Bitmap Heap Scan because sequential scans are suppressed. Its cost rises to 4370. 29..8357.69. It still returns 188,283 rows versus 188,112 estimated and visits all 1,636 heap blocks, with 1,868 shared hits and 493 reads overall. Planning takes 0.099 ms and execution 22.959 ms. This is slightly faster than D1 in this single run despite the higher estimated cost. It does not establish that bitmap access is generally better for this broad predicate; cache state and measurement variation affect elapsed time.

Screenshot Q3.2 (2)	Marks: /2
C3 uses Seq Scan after index, index-only and bitmap scans are suppressed. Its cost is 0. 00..4091.51, with 8,329 rows estimated and 8,158 returned. It discards 188,283 rows and has 1,636 shared hits. Planning takes 0.108 ms and execution 20.725 ms, slower than both bitmap C runs. Removing the selective index option forces a full-table read and filter. D3 also uses Seq Scan, with the same cost and row estimates as D1. Planning takes 0.095 ms and execution 21.383 ms. It returns 188,283 rows, discards 8,158 and has 1,636 shared hits. This remains the natural access method for a query needing nearly every row. Planner costs are estimates in cost units, not milliseconds. The comparison shows how row selectivity, page access and extra index work influence the chosen method. The switches discourage scan methods; they are useful for this experiment but should not replace normal planner choices in production. All four switches were restored to on.	
Comments:	

Screenshot Q4.1 (1)	Marks: /2

```
ssignment2=# \r
查询缓存区重置(已清空).
ssignment2=# \if :{?a2_right}
ssignment2@# \echo This is RIGHT. Run this block in the LEFT window.
忽略\echo命令; 使用\endif或Ctrl-C退出当前\if块
ssignment2@# \else
ssignment2=# \set PROMPT1 'LEFT %/%R%x%# '
EFT assignment2=# \set PROMPT2 'LEFT %/%R%x%# '
EFT assignment2=# \set a2_left true
EFT assignment2=#
EFT assignment2=# SELECT pg_backend_pid() AS session_pid,
EFT assignment2-#        current_database() AS database_name;
 session_pid | database_name
-------------+---------------
        8668 | assignment2
(1 行记录)

时间: 0.572 ms
EFT assignment2=#
EFT assignment2=# BEGIN;
EGIN
时间: 0.135 ms
EFT assignment2=*# SAVEPOINT collect_attempt;
AVEPOINT
时间: 0.086 ms
EFT assignment2=*#
EFT assignment2=*# SHOW transaction_isolation;
 transaction_isolation
-----------------------
 read committed
(1 行记录)

时间: 0.116 ms
EFT assignment2=*#
EFT assignment2=*# WITH latest_events AS (
EFT assignment2(*#     SELECT DISTINCT ON (vehicle_id)
EFT assignment2(*#         vehicle_id, event_id, member_id, event_type, event_time
EFT assignment2(*#     FROM BOOKING_EVENTS
EFT assignment2(*#     ORDER BY vehicle_id, event_time DESC, event_id DESC
EFT assignment2(*# )
EFT assignment2-*# SELECT vehicle_id, event_id, member_id, event_type, event_time
EFT assignment2-*# FROM latest_events
EFT assignment2-*# WHERE event_type = 'Return'
EFT assignment2-*# ORDER BY vehicle_id
EFT assignment2-*# LIMIT 1;
 vehicle_id | event_id | member_id | event_type |     event_time
------------+----------+-----------+------------+---------------------
 VEH-0004   |   193434 |      9529 | Return     | 2026-06-25 10:54:56
(1 行记录)

时间: 298.961 ms
EFT assignment2=*#
EFT assignment2=*# \endif
```

Comments:

Screenshot Q4.1 (2)	Marks: /0.5

```
GHT assignment2=#
GHT assignment2=#
GHT assignment2=#
GHT assignment2=#
GHT assignment2=#
GHT assignment2=#
GHT assignment2=#
GHT assignment2=# \r
询缓存区重置(已清空).
GHT assignment2=# \if :{?a2_right}
GHT assignment2=#
GHT assignment2=# BEGIN;
GIN
间: 0.303 ms
GHT assignment2=*#
GHT assignment2=*# SHOW transaction_isolation;
ransaction_isolation
---------------------
ead committed
 行记录)

间: 0.733 ms
GHT assignment2=*#
GHT assignment2=*# SELECT *
GHT assignment2-*# FROM (
GHT assignment2(*#     SELECT vehicle_id, event_id, member_id, event_type, event_time
GHT assignment2(*#     FROM BOOKING_EVENTS
GHT assignment2(*#     WHERE vehicle_id = 'VEH-0004'
GHT assignment2(*#     ORDER BY event_time DESC, event_id DESC
GHT assignment2(*#     LIMIT 1
GHT assignment2(*# ) AS latest_event
GHT assignment2-*# WHERE event_type = 'Return';
ehicle_id | event_id | member_id | event_type |     event_time
----------+----------+-----------+------------+---------------------
EH-0004   |   193434 |      9529 | Return     | 2026-06-25 10:54:56
 行记录)

间: 7.865 ms
GHT assignment2=*#
GHT assignment2=*# INSERT INTO BOOKING_EVENTS
GHT assignment2-*#     (member_id, vehicle_id, event_type, event_time, amount)
GHT assignment2-*# VALUES
GHT assignment2-*#     (2, 'VEH-0004', 'Reserve', NOW() + INTERVAL '3 days', NULL)
GHT assignment2-*# RETURNING event_id, member_id, vehicle_id, event_type, event_time;
vent_id | member_id | vehicle_id | event_type |         event_time
--------+-----------+------------+------------+----------------------------
 196446 |         2 | VEH-0004   | Reserve    | 2026-10-31 22:22:48.271341
 行记录)

SERT 0 1
间: 1.556 ms
GHT assignment2=*#
GHT assignment2=*# COMMIT;
MMIT
间: 0.272 ms
GHT assignment2=#
GHT assignment2=# \else
GHT assignment2@# \echo Please click the RIGHT window and paste this block again.
略\echo命令; 使用\endif或Ctrl-C退出当前\if块
GHT assignment2@# \endif
```

Comments:

Screenshot Q4.1 (3)	Marks: /1

```
(1 行记录)

时间: 0.116 ms
FT assignment2=*#
FT assignment2=*# WITH latest_events AS (
FT assignment2(*#     SELECT DISTINCT ON (vehicle_id)
FT assignment2(*#         vehicle_id, event_id, member_id, event_type, event_time
FT assignment2(*#     FROM BOOKING_EVENTS
FT assignment2(*#     ORDER BY vehicle_id, event_time DESC, event_id DESC
FT assignment2(*# )
FT assignment2-*# SELECT vehicle_id, event_id, member_id, event_type, event_time
FT assignment2-*# FROM latest_events
FT assignment2-*# WHERE event_type = 'Return'
FT assignment2-*# ORDER BY vehicle_id
FT assignment2-*# LIMIT 1;
ehicle_id | event_id | member_id | event_type |     event_time
-----------+----------+-----------+------------+---------------------
EH-0004   |   193434 |      9529 | Return     | 2026-06-25 10:54:56
(1 行记录)

时间: 298.961 ms
FT assignment2=*#
FT assignment2=*# \endif
FT assignment2=*# \r
查询缓存区重置(已清空).
FT assignment2=*# \if :{?a2_left}
FT assignment2=*#
FT assignment2=*# INSERT INTO BOOKING_EVENTS
FT assignment2-*#     (member_id, vehicle_id, event_type, event_time, amount)
FT assignment2-*# VALUES
FT assignment2-*#     (1, 'VEH-0004', 'Collect', NOW(), NULL);
错误:  Vehicle VEH-0004 is reserved by member 2
上下文:  在RAISE的第22行的PL/pgSQL函数udf_bi_collect()
时间: 10.551 ms
FT assignment2=!#
FT assignment2=!# COMMIT;
OLLBACK
时间: 0.233 ms
FT assignment2=#
FT assignment2=# SELECT
FT assignment2-#     event_id,
FT assignment2-#     member_id,
FT assignment2-#     vehicle_id,
FT assignment2-#     event_type,
FT assignment2-#     event_time,
FT assignment2-#     amount
FT assignment2-# FROM BOOKING_EVENTS
FT assignment2-# WHERE vehicle_id = 'VEH-0004'
FT assignment2-# ORDER BY event_time DESC, event_id DESC
FT assignment2-# LIMIT 3;
vent_id | member_id | vehicle_id | event_type |         event_time         | amount
---------+-----------+------------+------------+----------------------------+--------
  196446 |         2 | VEH-0004   | Reserve    | 2026-10-31 22:22:48.271341 |
  193434 |      9529 | VEH-0004   | Return     | 2026-06-25 10:54:56        |  67.11
  193396 |      9529 | VEH-0004   | Collect    | 2026-06-25 09:40:08        |
(3 行记录)

时间: 8.172 ms
FT assignment2=#
FT assignment2=# \else
FT assignment2@# \echo Please click the LEFT window and paste this block again.
略\echo命令; 使用\endif或Ctrl-C退出当前\if块
FT assignment2@# \endif
FT assignment2=#
FT assignment2=#
```

Comments:

Screenshot Q4.1 (4)	Marks: /2.5
The schedule is T1 BEGIN and availability read; T2 BEGIN and availability read; T2 Reserve and COMMIT; T1 Collect attempt; trigger exception; T1 ROLLBACK. T1 initially sees an available car, but that answer becomes stale before its next operation. At READ COMMITTED, each statement uses a fresh snapshot. The Collect attempt runs after T2 has committed, so the trigger can see the committed Reserve. The trigger orders events by event time and treats that future-dated reservation as the latest state. Because it belongs to member 2, it refuses a Collect by member 1. An ordinary SELECT reads without a row lock that reserves this car. Its ACCESS SHARE relation lock is compatible with the other transaction's insertion, so it does not prevent T2 from changing the answer. T1 stays open but does not retain its first statement's snapshot for later statements. In this schedule the rejection is a business-rule exception, with no circular lock wait. For the co-operative, the person at the parking bay is refused the car even though the other person reserved it three days ahead. The simplified trigger makes a future reservation an immediate hold. Coordinating availability checks and writes would address races, while interval-aware booking rules would also be needed to distinguish available now from reserved later.	
Comments:	

Screenshot Q4.2	Marks: /3

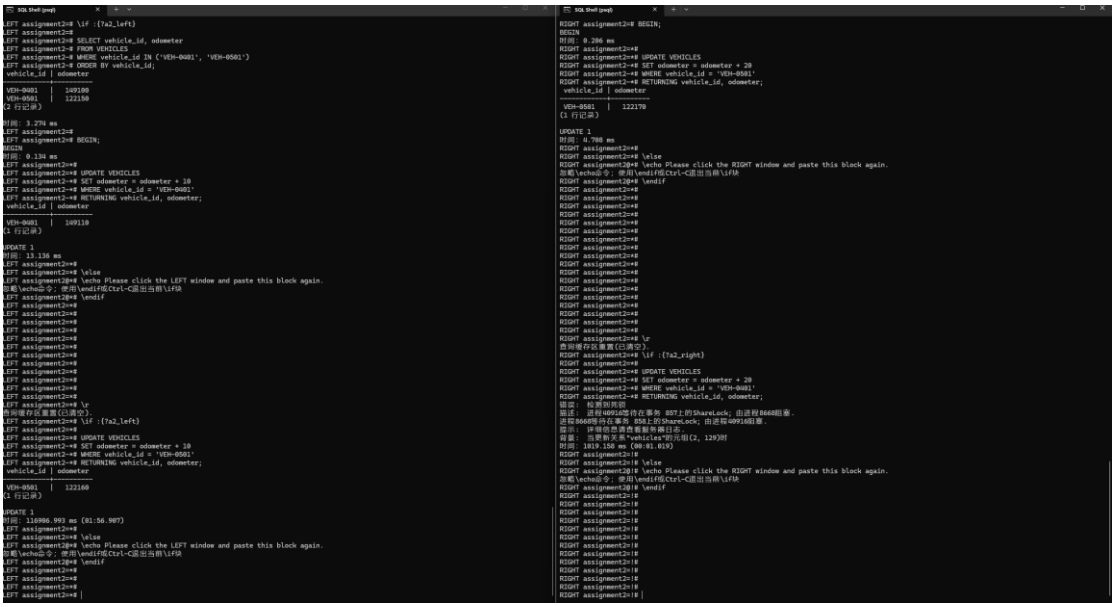


Screenshot Q4.2	Marks: /3

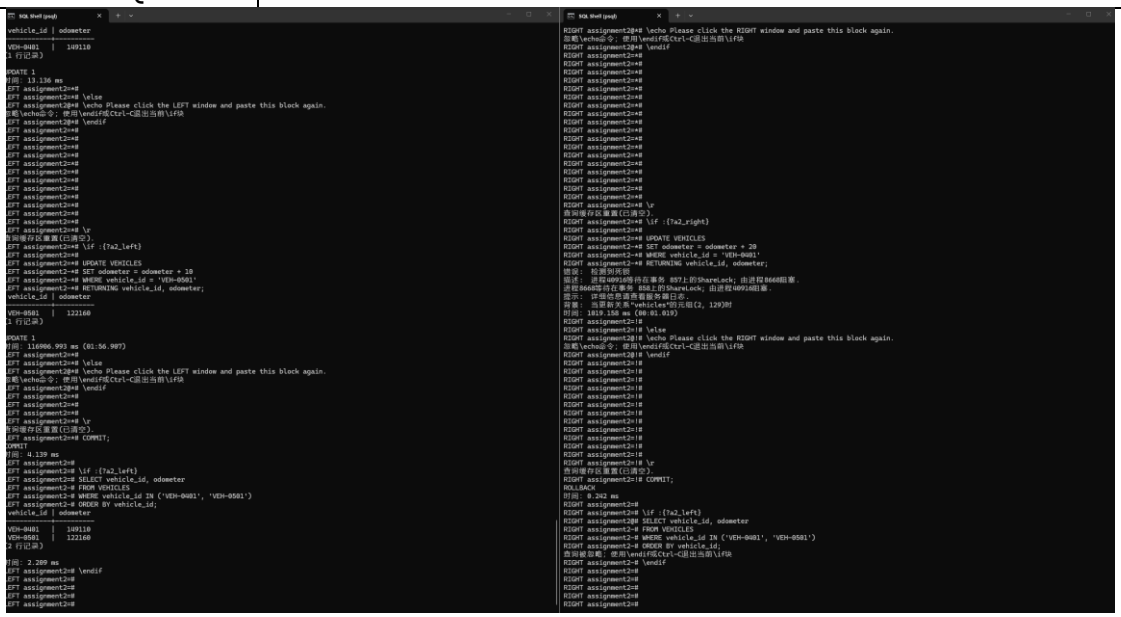
```
 VEH-0501    |   122150
(2 行记录)

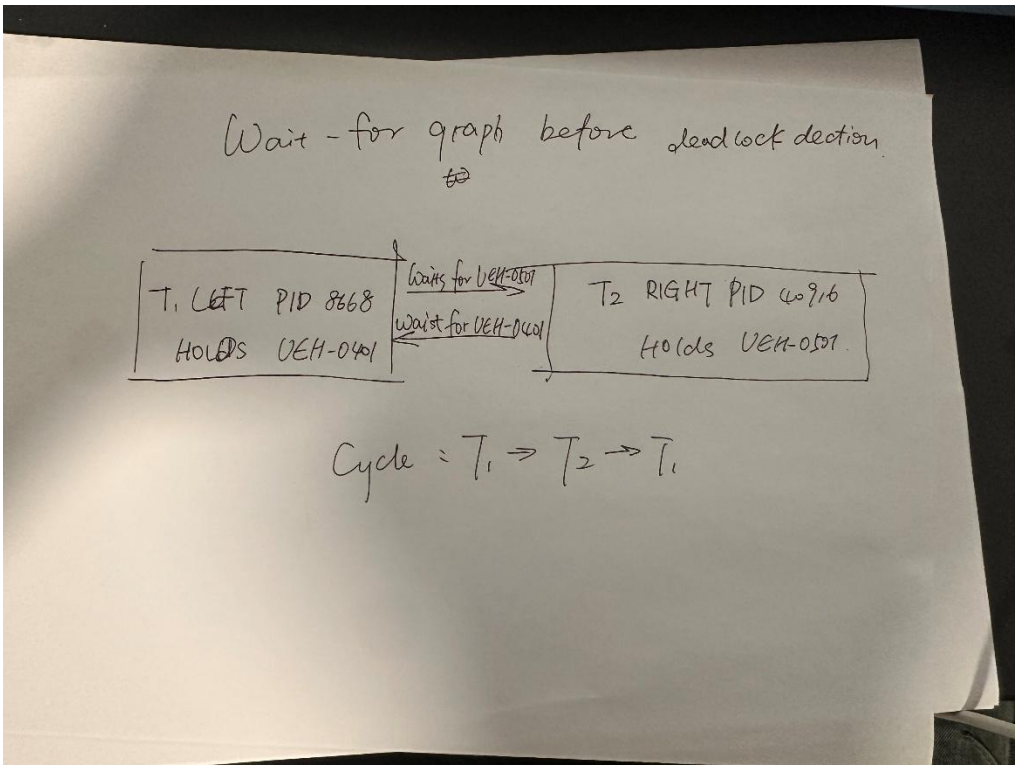
时间: 3.274 ms
LEFT assignment2=#
LEFT assignment2=# BEGIN;
BEGIN
时间: 0.134 ms
LEFT assignment2=*#
LEFT assignment2=*# UPDATE VEHICLES
LEFT assignment2-*# SET odometer = odometer + 10
LEFT assignment2-*# WHERE vehicle_id = 'VEH-0401'
LEFT assignment2-*# RETURNING vehicle_id, odometer;
 vehicle_id | odometer
------------+----------
 VEH-0401   |   149110
(1 行记录)

UPDATE 1
时间: 13.136 ms
LEFT assignment2=*#
LEFT assignment2=*# \else
LEFT assignment2@*# \echo Please click the LEFT window and paste this block again.
忽略\echo命令; 使用\endif或Ctrl-C退出当前\if块
LEFT assignment2@*# \endif
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*#
LEFT assignment2=*# \r
查询缓存区重置(已清空).
LEFT assignment2=*# \if :{?a2_left}
LEFT assignment2=*#
LEFT assignment2=*# UPDATE VEHICLES
LEFT assignment2-*# SET odometer = odometer + 10
LEFT assignment2-*# WHERE vehicle_id = 'VEH-0501'
LEFT assignment2-*# RETURNING vehicle_id, odometer;
```



Screenshot Q4.2	Marks: /3





Deadlock explanation

T1 updates VEH-0401 by 10 and holds its row lock. T2 updates VEH-0501 by 20 and holds the other row lock. T1 then requests VEH-0501 and waits. While it is waiting, T2 requests VEH-0401, completing the cycle. Both updates cannot finish until the other transaction ends.

PostgreSQL detects the deadlock and aborts T2 in this recorded run. T2's earlier +20 on VEH-0501 is undone and its locks are released. T1's waiting update resumes, returning 122160, and T1 commits. T2's attempted COMMIT reports ROLLBACK. Final values are VEH-0401 = 149110 and VEH-0501 = 122160.

Screenshot Q4.2	Marks: /3

The full original error is shown below and transcribed without translating its diagnostic text. T1's reported 116906.993 ms includes the time spent waiting while the next command was entered in the other window. T2's error took 1019.158 ms. These are not ordinary update processing times.

The wait-for graph immediately before detection has T1 -> T2 for VEH-0501 and T2 -> T1 for VEH-0401. Its cycle T1 -> T2 -> T1 explains why neither could proceed. The photographed hand drawing marks both held resources and both waiting edges.

A consistent ascending vehicle_id lock order would prevent this cycle: every correction transaction must acquire VEH-0401 before VEH-0501. A transaction blocked on the first row would then hold no later row needed by its predecessor. The co-operative would pay with more waiting or queuing when corrections overlap, while unrelated vehicles could still be processed concurrently. The victim can differ on a rerun, so the outcome here is based on the actual screenshot.

Full diagnostic text from the recorded error

错误: 检测到死锁

描述: 进程 40916 等待在事务 857 上的 ShareLock; 由进程 8668 阻塞.

进程 8668 等待在事务 858 上的 ShareLock; 由进程 40916 阻塞.

提示: 详细信息请查看服务器日志.

背景: 当更新关系"vehicles"的元组(2, 129)时

Technical references

PostgreSQL 18 Using EXPLAIN

PostgreSQL 18 Materialized Views

PostgreSQL 18 Indexes on Expressions

PostgreSQL 18 Explicit Locking and Deadlocks

Comments:

Screenshot P	Marks: /2
Comments:	

Penalty: _____

Reason of the Penalty

(Late Submission/No Show to Tutor/Purely AI Generated Solutions): _____

Final Marks: _____ / 30

---ooo000OOO000ooo---